

RAGHU ENGINEERING COLLEGE
(Autonomous)

**Affiliated to JNTU-GV-Vizianagaram Approved by AICTE, Accredited by
NBA, Accredited by NAAC with A+ grade**

www.raghuenggcollege.com

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project work entitled “**A LEARNING MODEL ON CUSTOMER CHURN FORECASTING FOR TELECOM PROVIDERS**” done by **SV Manikanta Kumar, Syed Asmath, T Rama Krishna, T Shalini** bearing regd no: **21981A05D9, 21981A05F9, 21981A05G5, 21981A05H1** are students of B.Tech in the Department of Computer Science and Engineering, Raghu Engineering College, during the period 2021-2025, in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering to the Jawaharlal Nehru Technological University- Gurajada, Vizianagaram is a record of bonafide work carried out under my guidance and supervision.

The results embodied in this Project report have not been submitted to any other University or Institute for the award of any degree.

INTERNAL GUIDE

Mr. A. A. Narasimham,
Dept of CSE,
Raghu Engineering College,
Dakamarri(V),
Visakhapatnam.

HEAD OF THE DEPARTMENT

Dr P Appala Naidu,
Dept of CSE,
Raghu Engineering College,
Dakamarri(V),
Visakhapatnam.

EXTERNAL EXAMINER

DISSERTATION APPROVAL SHEET

This is to certify that the dissertation titled

A LEARNING MODEL ON CUSTOMER CHURN FORECASTING FOR TELECOM PROVIDERS

Submitted By

S V Manikanta Kumar Syed Asmath

T Rama Krishna T Shalini

*Is approved for the degree of **Bachelor of Technology***

Mr. A. A. Narasimham (Guide)

Internal Examiner

External Examiner

HOD

Date:

DECLARATION

This to certify that this project entitled “**A LEARNING MODEL ON CUSTOMER CHURN FORECASTING FOR TELECOM PROVIDERS**” is bonafide work done by our team, in partial fulfillment of the requirements for the award of the degree B.Tech. and submitted to the Department of Computer Science & Engineering, **Raghu Engineering College, Dakamarri.**

We also declare that this project is a result of our own effort and that has not been copied from anyone and we have taken only citations from the sources which are mentioned in the references.

This work was not submitted earlier at any other University or Institute for the award of any degree.

PLACE:

DATE:

NAME:

S V Manikanta Kumar
Syed Asmath
T Ramakrishna
T Shalini

ACKNOWLEDGEMENT

We express our sincere gratitude to my esteemed Institute “Raghu Engineering College”, which has provided us with an opportunity to fulfill the most cherished desire to reach my goal.

We take this opportunity with great pleasure to put on record our ineffable personal indebtedness to **Sri Raghu Kalidindi, Chairman of Raghu Engineering College** for providing necessary departmental facilities.

We would like to thank **Dr. Ch. Srinivasu**, Principal, Raghu Engineering College, Visakhapatnam, **Dr. R. Siva Ranjani**- Dean CSE, **Dr. E. V. V. Ramanamurthy**- Controller of Examinations, and the Management of “**Raghu Engineering College**”, for providing the requisite facilities to carry them out the project in the campus.

Our sincere thanks to **Dr. P. Appala Naidu**, HOD, Department of Computer Science and Engineering, Raghu Engineering College, for this kind support in the successful completion of this work.

Our sincere thanks to **Mr. Ch. Seshadri Rao**, Project Coordinator, Department of Computer Science and Engineering, Raghu Engineering College, for this kind support in the successful completion of this work.

We sincerely express our deep sense of gratitude to **Mr. A. A. Narasimham**, Assistant Professor, Department of Computer Science and Engineering, Raghu Engineering College, for his perspicacity, wisdom, and sagacity coupled with compassion and patience. It is our great pleasure to submit this work under his wing.

We extend deep-hearted thanks to all faculty members of the Computer Science department for the value-based imparting of theory and practical subjects, which were used in the project.

We are thankful to the non-teaching staff of the Department of Computer Science and Engineering, Raghu Engineering College, for their inexpressible support.

Regards

S V Manikanta Kumar (21981A05D9)
Syed Asmath (21981A05F9)
T Rama Krishna (21981A05G5)
T Shalini (21981A05H1)

ABSTRACT

The technical infrastructure has expanded quickly, changing how businesses operate. One result of growing digitalization is subscription-based services, and as there are now more options for goods and services, customer churning has grown to be a significant issue and a threat to all businesses. Because it directly affects their earnings, businesses are striving to develop tools to predict potential client turnover. It is essential to discover the aspects that cause this churn in order to prevent client loss. Our work's key contribution is to prepare a churn prediction model that supports businesses identify the clients who experience churn. The customer data will undergo various data preprocessing phases before a classification technique is applied. We analyzed customer churn with classification techniques – Logistic Regression, Support Vector Classifier, Naïve Bayes, Decision Tree, Random Forest. These techniques are analyzed on various evaluation metrics such as – Accuracy, Precision, Recall, and F2 Score.

TABLE OF CONTENTS

CONTENT	PAGE NO
Certificate	i
Dissertation Approval Sheet	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Contents	vi
List of figures	viii
CHAPTER 1: INTRODUCTION	01
1.1 Introduction to Machine Learning	02
1.2 Existing System and Disadvantages	03
1.3 Proposed System and Advantages	03
1.4 Objectives of the Project	04
CHAPTER 2: LITERATURE SURVEY	05
2.1 Related Works	06
2.2 Survey Table	07
CHAPTER 3: SYSTEM ANALYSIS	08
3.1 Introduction	10
3.2 Use Case Diagrams	10
CHAPTER 4: SYSTEM REQUIREMENTS SPECIFICATIONS	17
4.1 Introduction of Requirements Analysis	18
4.2 Hardware & Software Requirements	18
4.3 Datasets	19
4.4 Software Requirements Specification	20
CHAPTER 5: SYSTEM DESIGN	23
5.1 Introduction	24
5.2 System Architecture	24
5.3 System Object Model	25

5.4	Object Description	25
5.5	Dynamic Model	27
5.6	Static Model	30
CHAPTER 6: IMPLEMENTATION		33
6.1	Software Used	34
6.2	Machine Learning	39
6.3	Algorithms	47
6.3.1	Logistic Regression	47
6.3.2	Support Vector Machine	48
6.3.3	Naïve Bayes algorithm	49
6.3.4	Decision Tree	51
6.3.5	Random Forest	52
6.4	Understanding dataset	53
6.5	Data preprocessing	55
6.6	Source Code	56
CHAPTER 7: SYSTEM TESTING		63
7.1	Levels of Testing	64
7.2	Test Cases	66
CHAPTER 8: SCREEN SHOTS		67
CHAPTER 9: SMOTE ANALYSIS		76
CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS		79
CHAPTER 11: REFERENCES		81

LIST OF FIGURES

DESCRIPTION	PAGE NO
Fig 3.2.1.1 Use case Diagram for Developer	11
Fig 3.2.2.1 Class Diagram	12
Fig 3.2.3.1 Activity Diagram	13
Fig 3.2.4.1 Sequence Diagram	14
Fig 3.2.5.1 Collaboration Diagram	15
Fig 3.2.6.1 Component Diagram	15
Fig 3.2.7.1 Deployment Diagram	16
Fig 5.2.1 System Architecture	24
Fig 5.4.1.2 Class Diagram for working of Application	27
Fig 5.5.1.1 State Chart Diagram	29
Fig 5.5.2.1 Sequence Diagram	30
Fig 5.6.1.1 Component Diagram	31
Fig 5.6.2.1 Deployment Diagram	32

CHAPTER-1

INTRODUCTION

1. INTRODUCTION

1.1 Introduction to Machine Learning

In the fast-paced landscape of today's business world, gaining a competitive edge is not just desirable; it's essential [1]. To thrive in this environment, companies must prioritize profit maximization. However, achieving this isn't solely about acquiring new customers or pushing new products. Rather, it involves retaining existing customers, which proves to be a more cost-effective strategy. Consequently, businesses must take deliberate steps to satisfy their customers and foster loyalty to ensure they continue utilizing their services.

Customer retention is pivotal as dissatisfied customers are prone to switching to competitors, resulting in substantial losses for the company. The term "customer churn" encapsulates this phenomenon, referring to when a customer terminates their subscription with one company to join another for various reasons. The primary drivers of churn include inadequate service, better pricing offered by competitors, and potential issues on the client's end.

Customer churn manifests in two forms: involuntary and voluntary. Involuntary churn occurs when customers are dissatisfied with factors such as non-payment, company policies, or product/service quality, prompting them to opt-out. On the other hand, voluntary churn can occur due to personal reasons like changes in financial circumstances or relocation, or deliberate decisions driven by technology preferences, service quality expectations, or psychological factors.

Addressing churn requires a proactive or reactive approach. In the reactive stance, companies respond after customers cancel their subscriptions, often resorting to incentives to retain them. However, in today's technologically advanced era, a proactive approach is more viable. By leveraging data analytics and predictive modeling, companies can forecast churn before it happens. This allows them to identify at-risk customers and take preemptive measures, such as offering tailored incentives or improving service quality, to enhance customer retention.

Data mining techniques play a pivotal role in this process, employing machine learning algorithms and statistical analyses to extract valuable insights from vast datasets [3] [4]. By categorizing customers into churners and non-churners, companies can effectively target the former group and implement strategies to mitigate churn. This involves employing classification tasks in machine learning, which encompass dimensionality reduction techniques like feature selection and extraction.

Furthermore, to address class imbalances in datasets, various balancing techniques such as SMOTE, Tomek Links, and Random Oversampling are employed [6] [7] [8]. The performance of predictive models is then evaluated using k-fold cross-validation and AUC curves, allowing companies to gauge the effectiveness of their churn prediction strategies.

In summary, effective management of customer churn requires a comprehensive approach that blends advanced analytics, machine learning, and proactive strategies. By anticipating and addressing churn through data-driven insights and targeted interventions, companies can bolster customer retention, enhance profitability, and maintain a competitive edge in the market.

1.2 Existing System and Disadvantages

Existing systems analysed customer churn using following classification techniques –

- Logistic Regression
- Support Vector Classifier
- Naïve Bayes
- Decision Tree

Disadvantages:

- Class imbalance problem is not handled
- Irrelevant features are not eliminated

1.3 Proposed System and Advantages

- We gather relevant data such as customer demographics, usage patterns, and churn history.
- Training the model with the pre-processed data will be done.
- We compare different classification algorithms for predicting the highest accuracy of the problem.
- Continuously monitor the performance of the model and updating it as necessary to adapt to changing customer behaviour and market conditions.
- Accuracy and efficiency of the model can be increased.

Advantages:

- We handle class imbalance problem; data resampling tool is widely used.
- Feature elimination using Variance Inflation Factor (VIF)

1.4 Objectives of the Project

- Conduct a systematic literature study on features influencing the customer churn in different companies.
- A comprehensive study of the proposals made by the researchers in identifying customer churn.
- Perform extensive feature engineering by comparing various class imbalance and dimensionality reduction techniques.
- Apply classification techniques in data mining and compare accuracy to arrive the best technique for prior identifying the customer churn.

CHAPTER-2

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Related Works

Wouter Verbeke and colleagues [9] identify predictive accuracy, comprehensibility, and justifiability as key aspects of customer churn prediction models. Their study compares advanced rule induction techniques, such as AntMiner+ and ALBA, with traditional methods like C4.5 and RIPPER. The results indicate that ALBA enhances classification learning, while AntMiner+ excels in accuracy, comprehensibility, and justifiability.

Shin-Yuan Hung et al. [3] conduct an empirical evaluation of data mining techniques, finding that decision trees and neural networks offer higher accuracy. Kristof Coussement et al. [10] focus on improving churn prediction by considering a wide range of customer types and selecting effective classification techniques. Their study reveals that Random Forest outperforms Logistic Regression and Support Vector Machines.

Nilam Nur Amir Sjarif et al. [11] propose an effective churn prediction model using the K-Nearest algorithm and Pearson correlation function. Their results demonstrate that K-Nearest achieves higher accuracy than other methods. E.W.T. Ngai et al. [4] suggest that data mining techniques can enhance CRM effectiveness, outlining steps to categorize research papers based on CRM frameworks and data mining applications.

Jonathan Burez et al. [12] discuss a case study involving a European pay TV company with a high customer churn rate. They apply various binary classification techniques and select the most suitable model. John Haden et al. [13] provide detailed steps to reduce customer churn and retain existing customers effectively.

Renato Alexandre de Lima Lemos et al. [14] compare supervised learning algorithms using cross-validation on a dataset from Brazilian banks, concluding that Random Forest offers higher accuracy. Soumi De et al. [2] review recent research on customer churn prediction, emphasizing the importance of precise evaluation measures and guidelines for model development.

Nhi N.Y.VO et al. [15] propose a customer churn prediction model using machine learning algorithms on call center data. Debajyoti Mukhopadhyay et al. [5] explore machine learning techniques for churn prediction and visualize results using graphs. Animesh Shukla [16] develops a web application for churn prediction using statistical techniques on bank data.

Victor Chimankpam Nwaogu et al. [1] propose a churn prediction model using various machine learning algorithms on Teldata dataset, with Neural Networks showing superior performance. Most existing solutions focus on imbalanced datasets, leading to high accuracy but low recall. This study aims to integrate multiple data balancing and reduction techniques to improve churn prediction efficiency.

2.2 Survey Table

S.No	Reference Paper Title	Purpose	Drawbacks
1	Customer churn prediction system: a machine learning approach.	A model is proposed with six phases using Adaboost and XGboost classifiers and got 81.71% and 80.8% accuracy.	Only two algorithms are considered and accuracy is not high.
2	Customer churn prediction in telecom using machine learning in big data platform	Build a system that predicts the churn of customers in SyriaTel Telecom Company. Four tree- based algorithms were chosen because of their diversity and applicability in this type of prediction. XGBOOST tree model achieved the best results in all measurements with 93.301%.	Only tree- based algorithms are considered.
3	Customer churn prediction using machine learning A study in the B2B subscription- based service context	Compared two ensemble learners, XGBoost and Random Forest, with a single base learner, Naïve Bayes.	Three algorithms are compared and not addressed class imbalance problem.

4	Applying Survival Analysis to Telecom Churn Data, American Journal of Theoretical and Applied Statistics	Proposed a solution that makes use of the survival analysis technique, which calculates the risks and chances of survival for consumers.	Considered the survival chances of consumers.
5	Binary Classification for Predicting Customer Churn	Showed that churn can be predicted with an accuracy of 84% with the Random Forest model.	Accuracy is not high.
6	The perils of proactive churn prevention using plan recommendations: evidence from a field experiment	Recommendations to overcome the threats of churn are addressed.	Conventional algorithms are used for prediction.

CHAPTER-3

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 Introduction

System analysis is the process of gathering and interpreting facts, diagnosing problems and using the information to recommend improvements on the system. System analysis is a problem-solving activity that requires intensive communication between the system users and system developers. System analysis is an important phase of any system development process. The system is studied to the minutest detail and analyzed. The system analyst plays the role of an interrogator and dwells deep into the working of the present system.

The system is viewed as a whole and the inputs to the system are identified. The outputs from the organization are traced through the various processing that the inputs phase through in the organization. A detailed study of these processes is made by various techniques like Interviews, reviews, etc.

The data collected by these sources is scrutinized to arrive to a conclusion. The conclusion is an understanding of how the system functions. This system is called the existing system.

3.2 UML Diagrams

3.2.1 Use Case diagram

Actors

- Developer
- user

Use case diagram is created to visualize the relationships between actors and use cases. A use case is a pattern of behavior the system exhibits. Each use case is a sequence of related transactions performed by an actor and the system. Diagrammatically actor and use case are represented by stick figure and oval respectively.

Symbols





Fig 3.2.1.1 Use case Diagram for Developer

3.2.2 Class diagram:

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely identify the class.

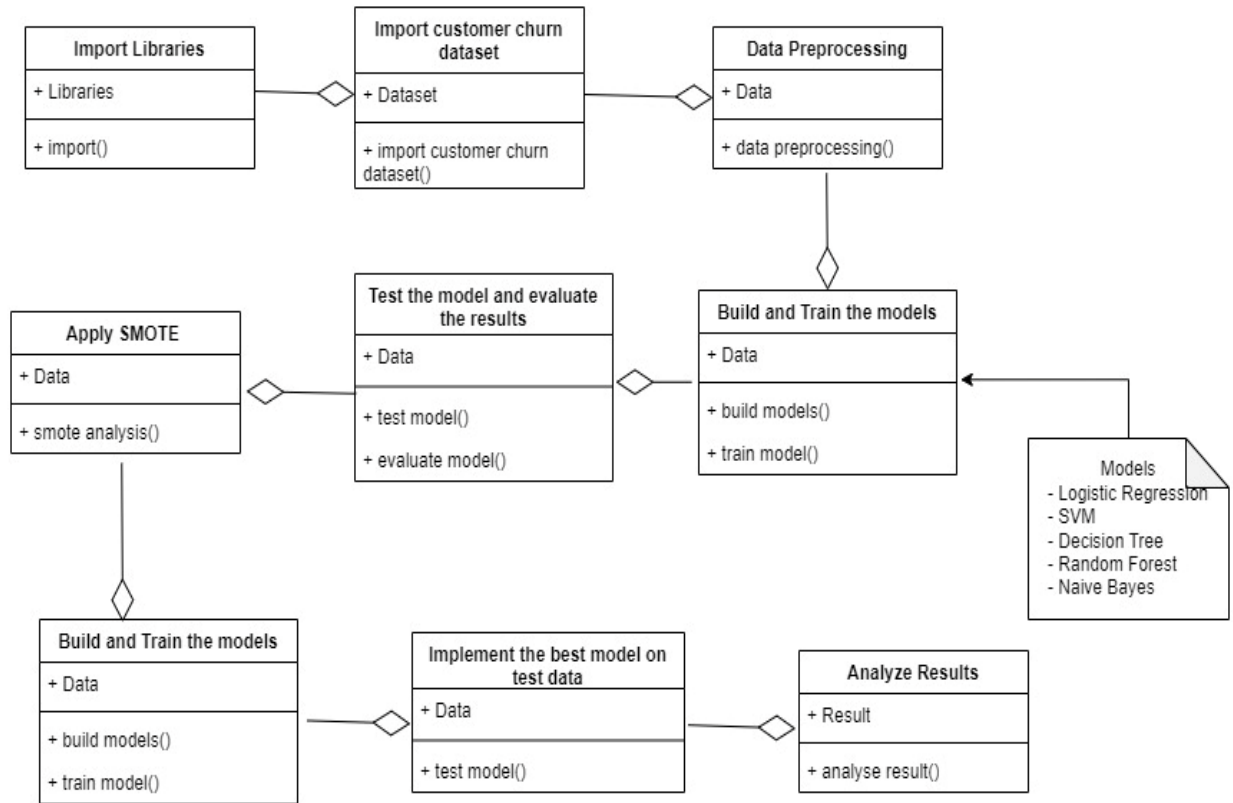


Fig 3.2.2.1 Class Diagram

3.2.3 Activity diagram:

The process flows in the system are captured in the activity diagram. Similar to a state diagram, an activity diagram also consists of activities, actions, transitions, initial and final states, and guard conditions.

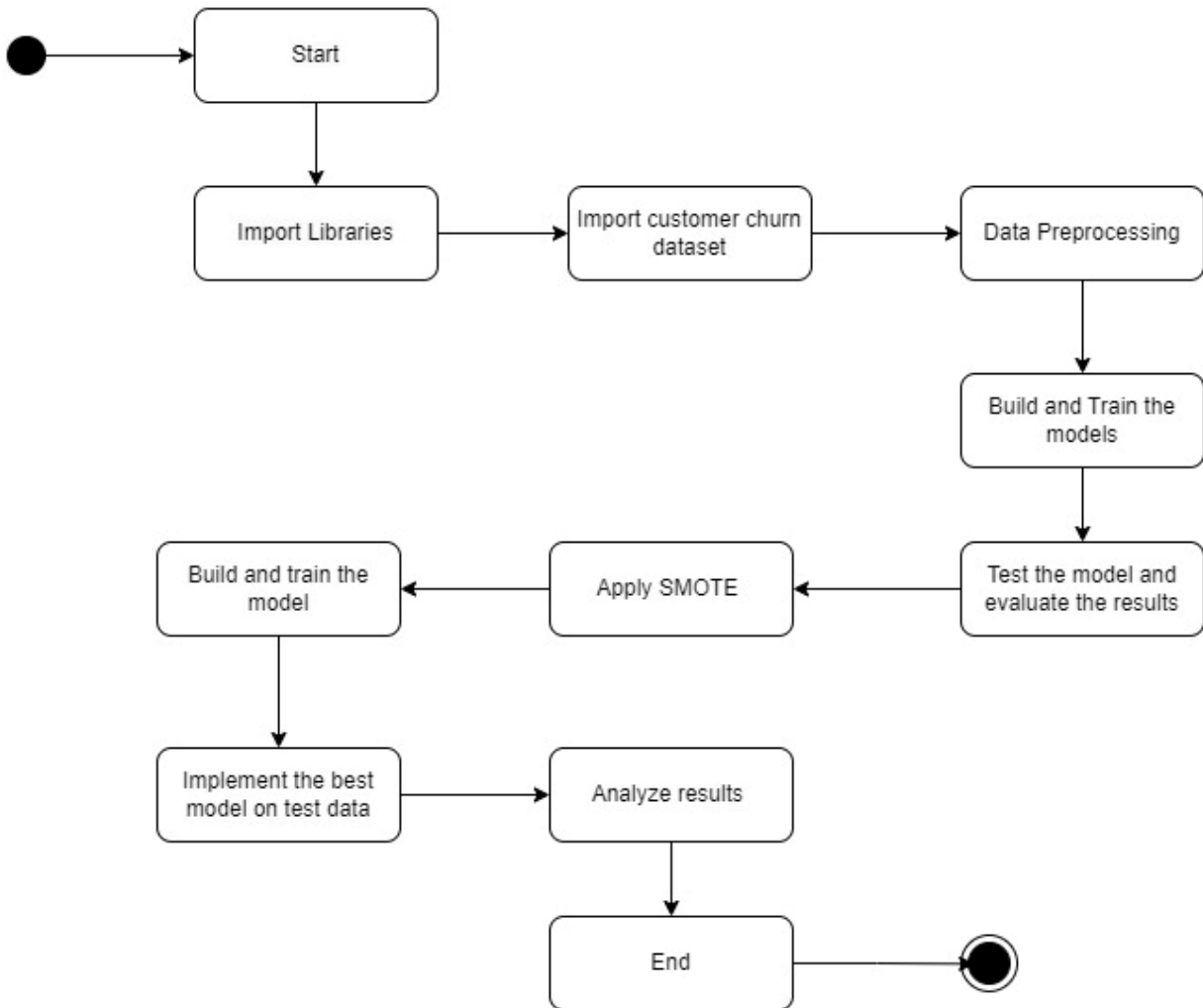


Fig 3.2.3.1 Activity Diagram

3.2.4 Sequence diagram:

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".

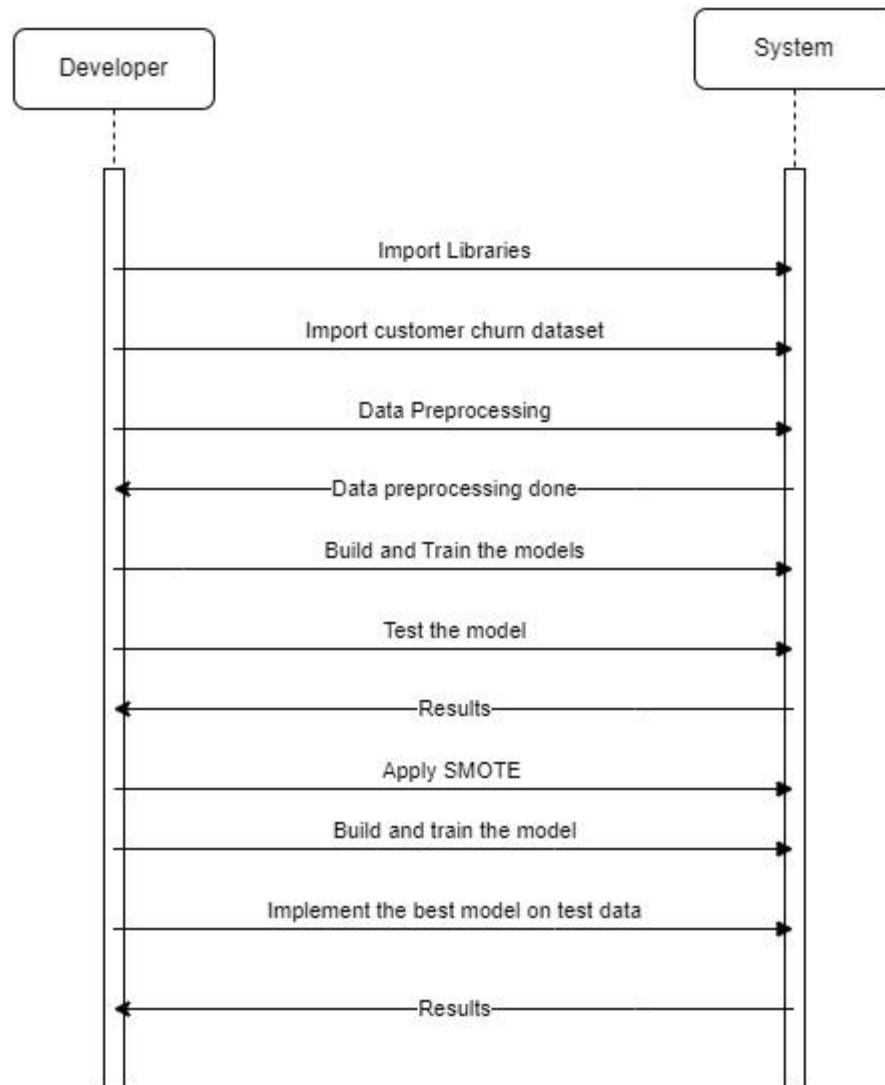


Fig 3.2.4.1 Sequence Diagram

3.2.5 Collaboration diagram:

A collaboration diagram groups together the interactions between different objects. The interactions are listed as numbered interactions that help to trace the sequence of the interactions. The collaboration diagram helps to identify all the possible interactions that each object has with other objects.

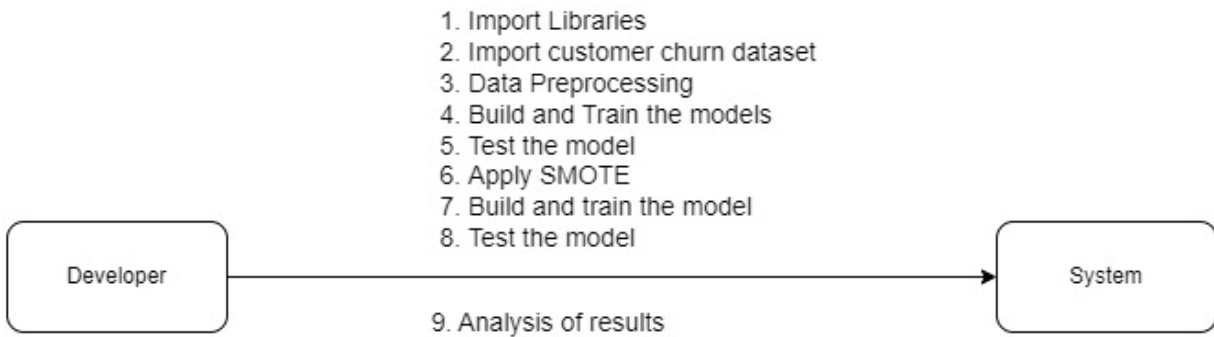


Fig 3.2.5.1 Collaboration Diagram

3.2.6 Component diagram:

The component diagram represents the high-level parts that make up the system. This diagram depicts, at a high level, what components form part of the system and how they are interrelated. A component diagram depicts the components culled after the system has undergone the development or construction phase.

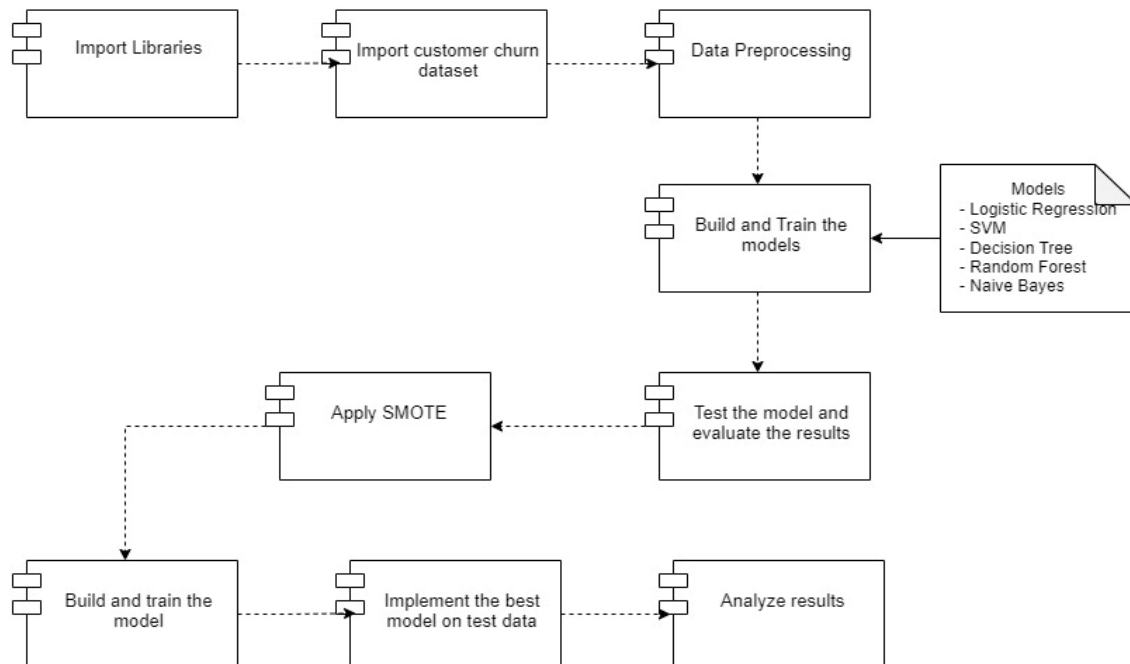


Fig 3.2.6.1 Component Diagram

3.2.7 Deployment diagram:

The deployment diagram captures the configuration of the runtime elements of the application. This diagram is by far most useful when a system is built and ready to be deployed.

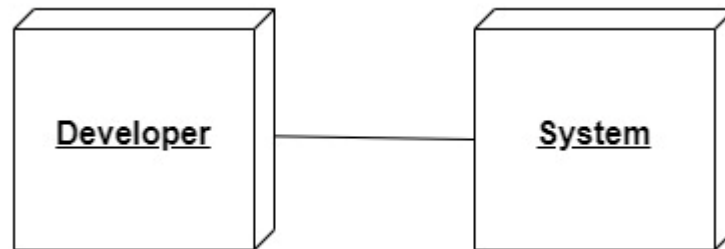


Fig 3.2.7.1 Deployment Diagram

CHAPTER-4
SYSTEM
REQUIREMENTS
SPECIFICATIONS

4. SYSTEM REQUIREMENTS SPECIFICATIONS

4.1 Introduction of Requirement Analysis

Requirements analysis, alternatively known as requirements engineering, involves the critical task of discerning user expectations for a new or updated product. These expectations, referred to as requirements, must meet specific criteria: they should be quantifiable, relevant, and detailed. This process holds significant importance within project management, serving as a cornerstone for project success. Requirements can encompass various aspects such as architectural, structural, behavioral, functional, and non-functional elements.

4.2 Hardware & Software Requirements

Hardware requirements

The most common set of requirements defined by any operating system or software application is the physical computer resources, also known as hardware. The following are the hardware interface used in this project.

- Processor: Intel Core i5 and above
- RAM: At least of 4GB
- HDD free space: Minimum 2 GB

Software requirements

Software requirements deal with defining software resource requirements and prerequisites that need to be installed on a computer to provide optimal functioning of an application. These requirements or prerequisites are generally not included in the software installation package and need to be installed separately before the software is installed.

- Operating system: Windows
- IDE: Jupiter / GOOGLE Colab
- Excel for dataset
- Python Packages:
 - Sklearn for ML algorithms
 - Seaborn for charts
 - Matplotlib for charts
 - Pandas to handle datasets

- Numpy for statistical values

4.3 Dataset

Each row represents a customer, each column contains customer's attributes described on the column Metadata. The data set includes information about:

- **Customers who left within the last month** – the column is called Churn
- **Services that each customer has signed up for** – phone, multiple lines, internet, online security, online backup, device protection, tech support, and streaming TV and movies
- **Customer account information** - how long they've been a customer, contract, payment method, paperless billing, monthly charges, and total charges
- **Demographic info about customers** – gender, age range, and if they have partners and dependents

RangeIndex: 7043 entries, 0 to 7042

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	customerID	7043 non-null	object
1	gender	7043 non-null	object
2	SeniorCitizen	7043 non-null	int64
3	Partner	7043 non-null	object
4	Dependents	7043 non-null	object
5	tenure	7043 non-null	int64
6	PhoneService	7043 non-null	object
7	MultipleLines	7043 non-null	object
8	InternetService	7043 non-null	object
9	OnlineSecurity	7043 non-null	object
10	OnlineBackup	7043 non-null	object
11	DeviceProtection	7043 non-null	object
12	TechSupport	7043 non-null	object
13	StreamingTV	7043 non-null	object
14	StreamingMovies	7043 non-null	object
15	Contract	7043 non-null	object
16	PaperlessBilling	7043 non-null	object
17	PaymentMethod	7043 non-null	object
18	MonthlyCharges	7043 non-null	float64
19	TotalCharges	7043 non-null	object
20	Churn	7043 non-null	object

dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Dataset dimension is: Rows – 7043 and Columns – 21

4.4 Software Requirements Specification

Software Requirement Specification (SRS) Format as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system. These requirements can be functional as well as non-functional depending upon the type of requirement. The interaction between different customers and contractors is done because it is necessary to fully understand the needs of customers.

4.4.1 Functional Requirements

In this, possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behavior of the system-which outputs should be produced from the given inputs. They describe the relationship between the input and output of the system. For each functional requirement, detailed description all the data inputs and their source, the units of measure, and the range of valid inputs must be specified.

4.4.2 Non-Functional Requirements

A non-functional requirement is a specification that describes the system's operation capabilities and constraints that enhance its functionality.

- Software Requirements
- Hardware Requirements
- Usability
- Reliability
- Performance
- Scalability
- Security
- Maintainability

4.4.2.1 User Interface and Human Factor

User interface design focuses on the following key areas:

- The design of interfaces between different software components.
- The design of interfaces between the software and other nonhuman producers and consumers of information.
- The design of the interface between a human and the computer.

4.4.2.2 Software Requirements

Software requirements deal with defining software resource requirements and prerequisites that need to be installed on a computer to provide optimal functioning of an application. These requirements or prerequisites are generally not included in the software installation package and need to be installed separately before the software is installed.

- Operating system: Windows
- IDE: Jupiter / GOOGLE Colab
- Excel for dataset
- Python Packages:
 - Sklearn for ML algorithms
 - Seaborn for charts
 - Matplotlib for charts
 - Pandas to handle datasets
 - Numpy for statistical values

4.4.2.3 Hardware Requirements

The most common set of requirements defined by any operating system or software application is the physical computer resources, also known as hardware. The following are the hardware interface used in this project.

- Processor: Intel Core i5 and above
- RAM: At least of 4GB
- HDD free space: Minimum 2 GB

4.4.2.4 Usability

- Simplified data visualization: The system presents the predictions in a clear and easy-to-understand format, such as graphs or charts.
- Customization options: The system allows the users to customize the prediction data they view, such as by selecting specific time period.
- Detailed documentation: The system provides detailed documentation on how to use the system and interpret the predictions.

4.4.2.5 Reliability

The system should be reliable, with minimal downtime or errors, to ensure consistent and dependable customer churn predictions.

4.4.2.6 Performance

- The system is able to predict prices with high accuracy.
- The system is able to handle a large volume of data and make predictions in a timely manner.
- The system should be able to handle a high number of concurrent users and requests.

4.4.2.6 Physical Environment

It is compatible with any system like a laptop & system.

4.4.2.7 Resource Requirements

Required software is to be installed like Python Idle, Jupyter notebook, and respected libraries are to be installed in our system to execute our project successfully.

4.4.2.8 Maintainability

- Robust documentation: Detailed documentation is essential for maintaining and updating a prediction system. This includes information about the system architecture, data sources, prediction models, and any other relevant details.
- Clear codebase: A clear and well-organized codebase makes it easier to maintain and update the system.

CHAPTER-5

SYSTEM DESIGN

5. SYSTEM DESIGN

5.1 Introduction

An effective system development life cycle (SDLC) should result in a high-quality system that meets customer expectations, reaches completion within time and cost evaluations, and works effectively and efficiently in the current and planned information technology infrastructure. System Development Life Cycle (SDLC) is a conceptual model which includes policies and procedures for developing or altering systems throughout their life cycles.

System design is the process of defining the architecture, models, interfaces, and data for a system to satisfy specified requirements. System design could be seen as the application of systems theory to product development. There is some overlap with the disciplines of system analysis, system architecture and system engineering.

The design phase describes how the system will fulfil the user requirements. To achieve this, we must create both logical and physical design. In this phase the system design functions and operations are described.

5.2 System Architecture

A system architecture or systems architecture is the conceptual model that defines the structure, behavior, and more views of a system. An architecture description and representation of a system, organized in a way that supports reasoning about structures and behavior of a system.

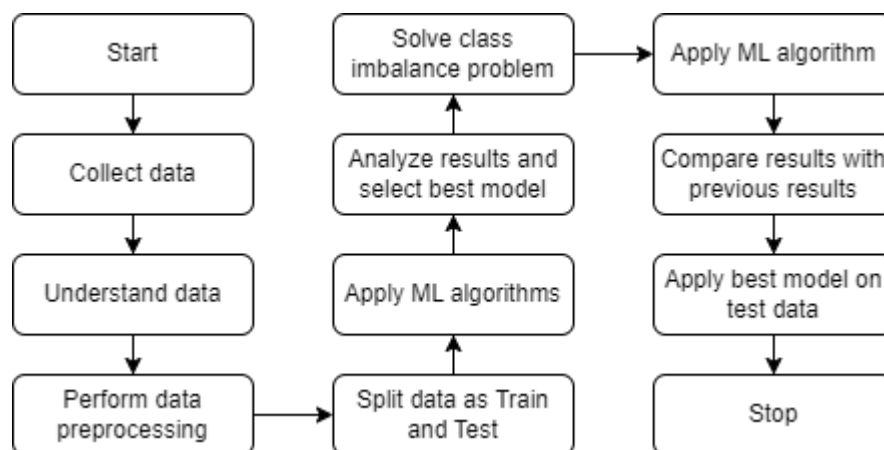


Fig 5.2.1 System Architecture

5.3 System Object Model

System object model (SOM) is an object- oriented library packaging technology developed by IBM that allows various programming languages to share class libraries, regardless of the language in which they were originally written. System object model was intended to be used as a solution to many of the interoperability and reuse problems that occur while sharing class libraries between object-oriented and non-object-oriented languages.

SOM was designed to be used across IBM's mainframe computers and desktops. It serves as an object-oriented model that can be distinguished from other models contained in object-oriented programming languages. SOM basically includes an interface definition a runtime environment with procedure calls and a set of enabling frameworks.

5.4 Object Description

Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams. Object diagrams so object diagrams are dependent upon class diagrams. Object diagrams represent an instance of a class diagram. The basic concepts are similar for class diagrams and object diagrams. Object diagrams also represent the static view of a system but this static view is a snapshot of a system at a particular moment. Object diagrams are used to render a set of objects and their relationships as an instance.

5.4.1 Objects

Object is any entity that can be manipulated by the commands of programming languages such as value, variable, function, or data structure.

5.4.2 Class Diagrams

A class diagram showing the systems classes, their attributes, operations, and collaborations and the relationships among objects. The class diagrams is the main building lock of object -oriented modelling. Class diagrams can also be used for data modelling. The classes in a class diagram represent both the main elements, interactions in the application and the classes to be programmed.

It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system.

The purpose of class diagram is to model the static view of an application. Class diagrams are the only diagrams which can be directly mapped with object -oriented languages and thus widely used at the time of construction.

UML diagrams like activity diagram, sequence diagram can only give the sequence flow of the application, however class diagram is a bit different. It is the most popular UML diagram in the coder community.

The purpose of the class diagram can be summarized as –

- Analysis and design of the static view of an application.
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering.

Class diagrams are the most popular UML diagrams used for construction of software applications. It is very important to learn the drawing procedure of class diagram.

Class diagrams have a lot of properties to consider while drawing but here the diagram will be considered from a top- level view.

Class diagram is basically a graphical representation of the static view of the system and represents different aspects of the application. A collection of class diagrams represents the whole system.

The following points should be remembered while drawing a class diagram –

- The name of the class diagram should be meaningful to describe the aspect of the system.
- Each element and their relationships should be identified in advance.
- Responsibility (attributes and methods) of each class should be clearly identified
- For each class, minimum number of properties should be specified, as unnecessary properties will make the diagram complicated.
- Use notes whenever required to describe some aspect of the diagram. At the end of the drawing, it should be understandable to the developer/coder.
- Finally, before making the final version, the diagram should be drawn on plain paper and reworked as many times as possible to make it correct.

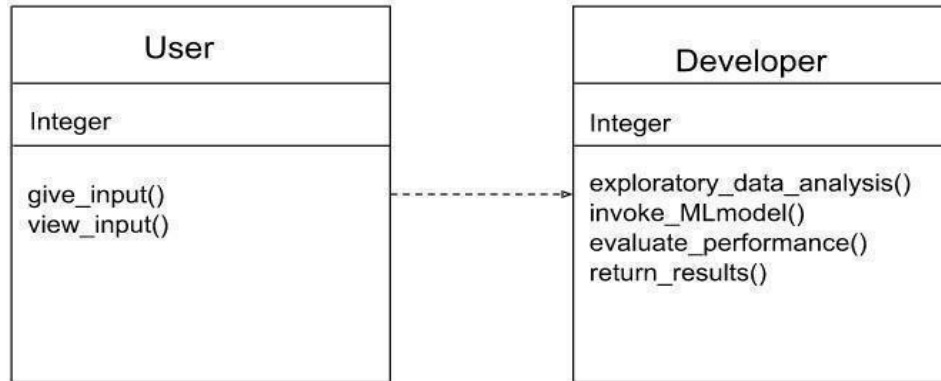


Fig 5.4.1.2 Class Diagram for working of Application

5.5 Dynamic Model

5.5.1 State Chart Diagram

An activity diagram is another important diagram in UML to describe the dynamic aspects of the system. An activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc. The basic purposes of activity diagrams are similar to the other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but the activity diagram is used to show message flow from one activity to another.

State Chart diagram is one of the five UML diagrams used to model the dynamic nature of a system. They define different states of an object during its lifetime and these states are changed by events. State Chart diagrams are useful to model the reactive systems. Reactive systems can be defined as a system that responds to external or internal events.

State Chart diagram describes the flow of control from one state to another state. States are defined as a condition in which an object exists and it changes when some event is triggered. The most important purpose of State Chart diagram is to model lifetime of an object from creation to termination.

State Chart diagrams are also used for forward and reverse engineering of a system. However, the main purpose is to model the reactive system.

Following are the main purposes of using State Chart diagrams

- To model the dynamic aspect of a system.
- To model the life time of a reactive system.
- To describe different states of an object during its life time.
- Define a state machine to model the states of an object.

State Chart diagram defines the states of a component and these state changes are dynamic in nature. Its specific purpose is to define the state changes triggered by events. Events are internal or external factors influencing the system.

State Chart diagrams are used to model the states and also the events operating on the system. When implementing a system, it is very important to clarify different states of an object during its life time and State Chart diagrams are used for this purpose. When these states and events are identified, they are used to model it and these models are used during the implementation of the system.

If we look into the practical implementation of State Chart diagram, then it is mainly used to analyze the object states influenced by events. This analysis is helpful to understand the system behavior during its execution.

The main usage can be described as –

- To model the object states of a system.
- To model the reactive system. Reactive system consists of reactive objects.
- To identify the events responsible for state changes.
- Forward and reverse engineering.

State Chart diagram is used to describe the states of different objects in its life cycle. Emphasis is placed on the state changes upon some internal or external events. These states of objects are important to analyze and implement them accurately.

State Chart diagrams are very important for describing the states. States can be identified as the condition of objects when a particular event occurs.

Before drawing a State Chart diagram, we should clarify the following points

- Identify the important objects to be analysed.
- Identify the states.
- Identify the events

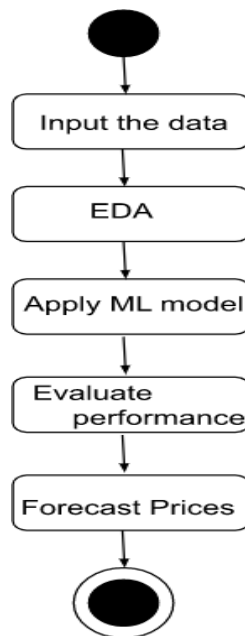


Fig 5.5.1.1 State Chart Diagram

5.5.2 Sequence diagram

A sequence diagram is a type of interaction diagram because it describes how and in what order a group of objects works together. A sequence diagram shows object interactions arranged in time sequence. The interaction that takes place in collaboration that either realizes a use case or an operation. Sequence diagrams provide high level interactions between user of the system and the system, between the system and the other systems, or between subsystems.

Sequence diagrams can be useful references for businesses and other organizations. Try drawing a sequence diagram to:

- Represent the details of a UML use case.
- Model the logic of a sophisticated procedure, function, or operation.
- See how objects and components interact with each other to complete a process.
- Plan and understand the detailed functionality of an existing or future scenario.

The following scenarios are ideal for using a sequence diagram:

- Usage scenario: A usage scenario is a diagram of how your system could potentially be used. It's a great way to make sure that you have worked through the logic of every usage scenario for the system.

- Method logic: Just as you might use a UML sequence diagram to explore the logic of a use case, you can use it to explore the logic of any function, procedure, or complex process.
- Service logic: If you consider a service to be a high-level method used by different clients, a sequence diagram is an ideal way to map that out.
- Sequence diagram Visio - Any sequence diagram that you create with Visio can also be uploaded into Lucid chart. Lucid chart supports .vsd and .vdx file import and is a great Microsoft Visio alternative. Almost all of the images you see in the UML section of this site were generated using Lucid chart.

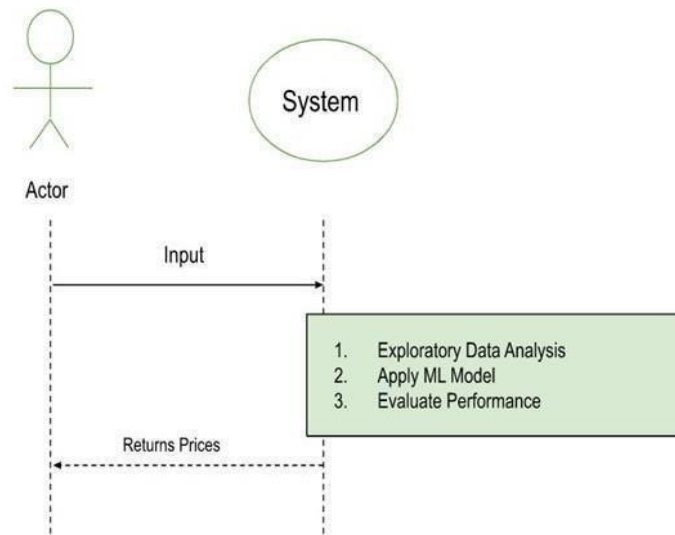


Fig 5.5.2.1 Sequence Diagram

5.6 Static Model

5.6.1 Component Diagram

UML Component diagrams are used in modelling the physical aspects of object- oriented systems that are used for visualizing, specifying, and documenting component- based systems and also for constructing executable systems through forward and reverse engineering.

Component diagram is a special kind of diagram in UML. The purpose is also different from all other diagrams discussed so far. It does not describe the functionality of the system but it describes the components used to make those functionalities.

Thus, from that point of view, component diagrams are used to visualize the physical components in a system. These components are libraries, packages, files, etc.

Component diagrams can also be described as a static implementation view of a system. Static implementation represents the organization of the components at a particular moment.

A single component diagram cannot represent the entire system but a collection of diagrams is used to represent the whole.

The purpose of the component diagram can be summarized as –

- Visualize the components of a system.
- Construct executables by using forward and reverse engineering.
- Describe the organization and relationships of the components.

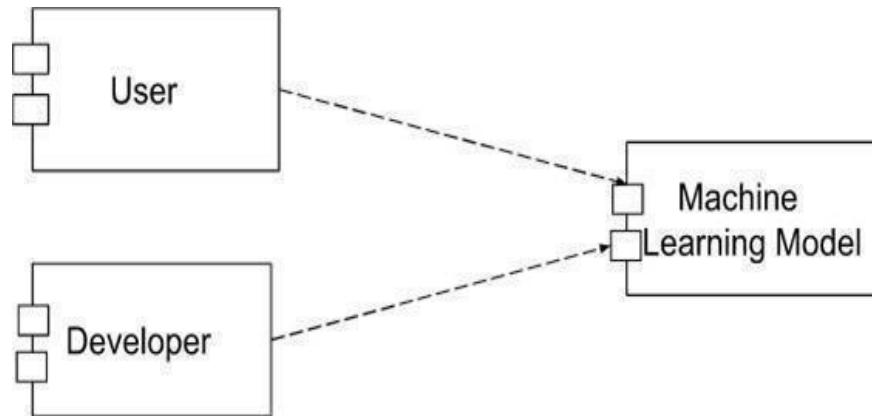


Fig 5.6.1.1 Component Diagram

5.6.2 Deployment Diagram

Deployment diagrams are used to visualize the topology of the physical components of a system, where the software components are deployed. Deployment diagrams are used to describe the static deployment view of a system. Deployment diagrams consist of nodes and their relationships.

Deployment diagrams are important for visualizing, specifying, and documenting embedded, client/server, and distributed systems and also for managing executable systems through forward and reverse engineering.

A deployment diagram is just a special kind of class diagram, which focuses on a system's nodes. Graphically, a deployment diagram is a collection of vertices and arcs. Deployment diagrams commonly contain:

- Nodes
- 3-D box represents a node, either software or hardware.

A UML deployment diagram is a diagram that shows the configuration of run time processing nodes and the components that live on them. Deployment diagrams is a kind of structure diagram used in modeling the physical aspects of an object-oriented system. They are often be used to model the static deployment view of a system (topology of the hardware).

Deployment diagrams are important for visualizing, specifying, and documenting embedded, client/server, and distributed systems and also for managing executable systems through forward and reverse engineering.

A deployment diagram is just a special kind of class diagram, which focuses on a system's nodes. Graphically, a deployment diagram is a collection of vertices and arcs.

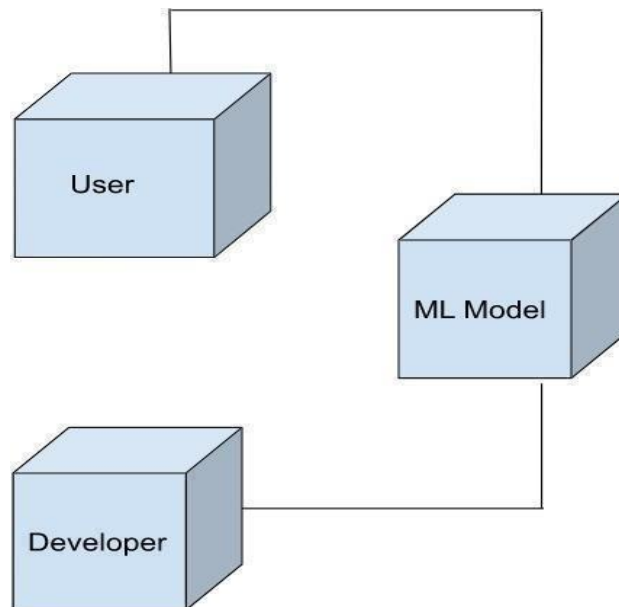


Fig 5.6.2.1 Deployment Diagram

CHAPTER-6

IMPLEMENTATION

6. IMPLEMENTATION

6.1 Software Used

6.1.1 Jupyter

Jupyter is an open-source web-based tool used for creating and sharing live code, equations, visualizations, and narratives in the form of notebooks. It provides an interactive environment for data analysis, machine learning, and scientific computing, and supports multiple programming languages such as Python, R, and Julia. Jupyter notebooks allow users to write code in cells, execute them individually or collectively, and visualize results in the form of graphs, charts, and tables. Jupyter is commonly used in data science, research, and collaborative computing environment.

Features of Jupyter

Jupyter provides a wide range of features that make it a popular tool for data analysis, machine learning, scientific computing, and research. Some of the key features of Jupyter include

- **Interactive notebooks**

Jupyter allows users to create and edit notebooks, which are documents that combine code, documentation, equations, visualizations, and other elements in an interactive and executable format. Notebooks are organized into cells, where users can write and execute code, view results, and document their findings.

- **Support for multiple programming languages**

Jupyter supports multiple programming languages, including Python, R, Julia, and many others. This allows users to write code in their language of choice and switch between different kernels (language-specific interpreters or execution environments) within the same notebook.

- **Code execution**

Jupyter allows users to write code in cells and execute them individually or collectively. This provides an interactive environment for data analysis, where users can experiment with code snippets, see results in real-time, and iterate on their analyses.

- **Rich media integration**

Jupyter supports the integration of rich media elements such as images, videos, interactive visualizations, and LaTeX equations. This allows users to create visually appealing and interactive

documents that combine code, results, and explanations.

- **Data visualization**

Jupyter provides built-in support for creating data visualizations using popular libraries such as Matplotlib, Seaborn, Plotly, and others. Users can generate various types of plots, charts, and graphs to explore and analyze data within the notebook itself.

- **Documentation and collaboration**

Jupyter allows users to document their work using Markdown, a popular markup language, within notebook cells. This allows for creating formatted text, explanations, and annotations to accompany code and results. Jupyter also supports collaboration, allowing users to share notebooks with others and work together in real-time.

- **Extensibility**

Jupyter is highly extensible, with a large ecosystem of extensions, plugins, and widgets available. This allows users to customize their Jupyter environment, add additional functionality, and integrate with other tools and libraries.

- **Reproducible research**

Jupyter promotes reproducible research by providing an interactive and executable format for documenting code, data, and results. Notebooks can be saved and shared, allowing others to reproduce and verify the findings.

6.1.2 Python programming language

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on

in the same Interview, Guido van Rossum continued: “I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC’s better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers.”

Python is a widely used general-purpose, high level programming language. It was created by Guido van Rossum in 1991 and further developed by the Python Software Foundation. It was designed with an emphasis on code readability, and its syntax allows programmers to express their concepts in fewer lines of code. Python is a programming language that lets you work quickly and integrate systems more efficiently.

There are two major Python versions: Python2 and Python3

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

The biggest strength of Python is huge collection of standard libraries which can be used for the following

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

There are more advantages of using Python programming language. Some of them are:

1. Extensive Libraries

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print „Hello World“. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms.

While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It comes with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

12. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

13. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

14. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonelle.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open Data Base Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

6.2 Machine Learning

Machine learning (ML) is a type of artificial intelligence (AI) that allows software applications to become more accurate at predicting outcomes without being explicitly programmed to do so. Machine learning algorithms use historical data as input to predict new output values.

Machine learning is important because it gives enterprises a view of trends in customer behavior and business operational patterns, as well as supports the development of new products. Many of today's leading companies, such as Facebook, Google and Uber, make machine learning a central part of their operations. Machine learning has become a significant competitive

differentiator for many companies.

Classical machine learning is often categorized by how an algorithm learns to become more accurate in its predictions. There are four basic approaches: supervised learning, unsupervised learning, semi-supervised learning and reinforcement learning. The type of algorithm data scientists choose to use depends on what type of data they want to predict

- **Supervised learning**

In this type of machine learning, data scientists supply algorithms with labeled training data and define the variables they want the algorithm to assess for correlations. Both the input and the output of the algorithm is specified.

- **Unsupervised learning**

This type of machine learning involves algorithms that train on unlabeled data. The algorithm scans through data sets looking for any meaningful connection. The data that algorithms train on as well as the predictions or recommendations they output are predetermined.

- **Semi-supervised learning**

This approach to machine learning involves a mix of the two preceding types. Data scientists may feed an algorithm mostly labeled training data, but the model is free to explore the data on its own and develop its own understanding of the data set.

- **Reinforcement learning**

Data scientists typically use reinforcement learning to teach a machine to complete a multi-step process for which there are clearly defined rules. Data scientists program an algorithm to complete a task and give it positive or negative cues as it works out how to complete a task. But for the most part, the algorithm decides on its own what steps to take along the way.

What is Machine Learning?

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of building models of data.

Fundamentally, machine learning involves building mathematical models to help understand data. “Learning” enters the fray when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be “learning” from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I’ll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based “learning” is similar to the “learning” exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we’ll discuss here.

Cross Validation

Learning the parameters of a prediction function and testing it on the same data is a methodological mistake: a model that would just repeat the labels of the samples that it has just seen would have a perfect score but would fail to predict anything useful on yet-unseen data. This situation is called overfitting. To avoid it, it is common practice when performing a (supervised) machine learning experiment to hold out part of the available data as a test set X-test, y-test. Note that the word “experiment” is not intended to denote academic use only, because even in commercial settings machine learning usually starts out experimentally. Here is a flowchart of typical cross validation workflow in model training. The best parameters can be determined by grid search techniques.

When evaluating different settings (“hyperparameters”) for estimators, such as the C setting that must be manually set for an SVM, there is still a risk of overfitting on the test set because the parameters can be tweaked until the estimator performs optimally. This way, knowledge about the test set can “leak” into the model and evaluation metrics no longer report on generalization performance. To solve this problem, yet another part of the dataset can be held out as a so-called “validation set”: training proceeds on the training set, after which evaluation is done on the validation set, and when the experiment seems to be successful, final evaluation can be done on the test set.

However, by partitioning the available data into three sets, we drastically reduce the number of samples which can be used for learning the model, and the results can depend on a particular random choice for the pair of (train, validation) sets. A solution to this problem is a

procedure called cross-validation (CV for short).

A test set should still be held out for final evaluation, but the validation set is no longer needed when doing CV. In the basic approach, called k-fold CV, the training set is split into k smaller sets (other approaches are described below, but generally follow the same principles). The following procedure is followed for each of the k “folds”:

- A model is trained using k-1 of the folds as training data;
- The resulting model is validated on the remaining part of the data (i.e., it is used as a test set to compute a performance measure such as accuracy). The performance measure reported by k-fold cross-validation is then the average of the values computed in the loop. This approach can be computationally expensive, but does not waste too much data (as is the case when fixing an arbitrary validation set), which is a major advantage in problems such as inverse inference where the number of samples is very small.

Challenges in Machine Learning

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as a whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

1. Quality of data

Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

2. Time-Consuming task

Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

3. Lack of specialist persons

As ML technology is still in its infancy stage, availability of expert resources is a tough job.

4. No clear objective for formulating business problems

Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

5. Issue of overfitting & underfitting

If the model is overfitting or underfitting, it cannot be represented well for the problem.

6. Curse of dimensionality

Another challenge ML model faces is too many features of data points. This can be a real hindrance.

7. Difficulty in deployment

Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machine Learning

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach.

Following are some real-world applications of ML

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

How to Start Learning Machine Learning?

Arthur Samuel coined the term “Machine Learning” in 1959 and defined it as a “Field of study that gives computers the capability to learn without being explicitly programmed”. And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to Indeed, Machine Learning Engineer is the best job of 2019 with a 344% growth and an average base salary of \$146,085 per year.

But there is still a lot of doubt about what exactly Machine Learning is and how to start

learning it? So this article deals with the Basics of Machine Learning and also the path you can follow to eventually become a full-fledged Machine Learning Engineer. Now let's get started!!!

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on mathematics as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc.

Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs (features) and its expected outputs (labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output (label).

Advantages of Machine learning

1. Easily identifies trends and patterns

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus software's, they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As ML algorithms gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an

algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

How does supervised machine learning work?

Supervised machine learning requires the data scientist to train the algorithm with both labeled inputs and desired outputs. Supervised learning algorithms are good for the following tasks

- **Binary classification:** Dividing data into two categories.
- **Multi-class classification:** Choosing between more than two types of answers.
- **Regression modeling:** Predicting continuous values.
- **Ensembling:** Combining the predictions of multiple machine learning models to produce an accurate prediction.

How does unsupervised machine learning work?

Unsupervised machine learning algorithms do not require data to be labeled. They sift through unlabeled data to look for patterns that can be used to group data points into subsets. Most types of deep learning, including neural networks, are unsupervised algorithms. Unsupervised learning algorithms are good for the following tasks

- **Clustering:** Splitting the dataset into groups based on similarity.
- **Anomaly detection:** Identifying unusual data points in a data set.
- **Association mining:** Identifying sets of items in a data set that frequently occur together.
- **Dimensionality reduction:** Reducing the number of variables in a data set.

6.3 Algorithms

6.3.1 Logistic Regression

Logistic regression is a statistical method used for binary classification tasks, where the output variable (dependent variable) is categorical and has only two possible outcomes, typically coded as 0 and 1. It's a type of regression analysis that is well-suited for predicting the probability

of a certain event occurring based on one or more predictor variables.

Here's a brief overview of how logistic regression works:

1. Sigmoid Function: Logistic regression uses the logistic function, also known as the sigmoid function, to model the relationship between the predictor variables and the probability of the outcome. The sigmoid function maps any real-valued number to a value between 0 and 1, which is useful for representing probabilities. The sigmoid function is defined as:

$$f(z) = \frac{1}{1+e^{-z}}$$

Where z is the linear combination of predictor variables and their respective coefficients.

2. Model Training: During the training phase, the logistic regression model learns the optimal values for the coefficients (weights) that minimize the error between the predicted probabilities and the actual outcomes in the training data. This is typically done using optimization algorithms like gradient descent.

3. Prediction: Once the model is trained, it can be used to predict the probability of the outcome for new data points. The predicted probability is then converted into a binary prediction using a threshold (usually 0.5). For example, if the predicted probability is greater than 0.5, the outcome is predicted as 1; otherwise, it's predicted as 0.

4. Evaluation: Logistic regression models can be evaluated using various metrics such as accuracy, precision, recall, F1-score, and ROC-AUC (Receiver Operating Characteristic – Area Under the Curve).

Logistic regression is widely used in various fields such as healthcare (e.g., predicting the likelihood of disease), marketing (e.g., predicting customer churn), finance (e.g., predicting loan defaults), and many others, where binary classification problems are common. It's a simple yet powerful algorithm that is easy to interpret and implement. However, it's important to note that logistic regression assumes a linear relationship between the predictor variables and the log-odds of the outcome, which may not always hold true in practice.

6.3.2 Support Vector Machine

A Support Vector Classifier (SVC), also known as Support Vector Machine (SVM) for classification, is a supervised learning algorithm used for classification tasks. Unlike logistic regression, which focuses on finding the best-fitting line (or hyperplane in higher dimensions), SVC aims to find the hyperplane that best separates the data into different classes while maximizing the

margin, which is the distance between the hyperplane and the nearest data points (support vectors). Here's how an SVC works:

- 1. Data Representation:** Like other classification algorithms, SVC requires labeled training data, where each data point is associated with a class label.
- 2. Feature Space Transformation (Optional):** In some cases, SVC might transform the feature space to a higher dimension using a technique called the kernel trick. This allows SVC to find a nonlinear decision boundary in the original feature space. Common kernels include linear, polynomial, radial basis function (RBF), and sigmoid.
- 3. Maximizing Margin:** SVC aims to find the hyperplane that maximizes the margin between the support vectors of different classes. The support vectors are the data points that lie closest to the decision boundary. By maximizing the margin, SVC aims to improve its generalization performance on unseen data.
- 4. Optimization:** The optimization problem in SVC involves finding the optimal hyperplane parameters (weights and bias) that minimize the classification error while maximizing the margin. This is typically formulated as a convex optimization problem and solved using techniques like quadratic programming.
- 5. Classification:** Once the optimal hyperplane is found, SVC can classify new data points by determining which side of the hyperplane they fall on. If a data point lies on one side of the hyperplane, it's classified as belonging to one class; if it lies on the other side, it's classified as belonging to the other class.

SVC is a powerful algorithm that can effectively handle high-dimensional data and complex decision boundaries. It's widely used in various domains, including image classification, text classification, bioinformatics, and finance. However, SVC's performance can be sensitive to the choice of hyperparameters such as the regularization parameter (C) and the kernel parameters. Proper parameter tuning and handling of imbalanced datasets are crucial for achieving good performance with SVC. Additionally, SVCs can become computationally expensive when dealing with large datasets.

6.3.3 Naïve Bayes algorithm

Naïve Bayes classification is a simple and effective probabilistic classification algorithm based on Bayes' theorem with the "naïve" assumption of conditional independence between features.

Despite its simplicity, Naïve Bayes classifiers often perform well in practice and are particularly useful when dealing with high-dimensional data.

Here's how Naïve Bayes classification works:

1. Bayes' Theorem: At the core of Naïve Bayes classification is Bayes' theorem, which describes the probability of a hypothesis (class label) given the evidence (features). Mathematically, it is represented as:

$$P(Y|X) = \frac{P(X|Y) \times P(Y)}{P(X)}$$

Where:

- $P(Y|X)$ is the posterior probability of class Y given the features X .
- $P(X|Y)$ is the likelihood of observing the features X given class Y .
- $P(Y)$ is the prior probability of class Y .
- $P(X)$ is the probability of observing the features X (which acts as a normalization factor).

2. Naïve Assumption: Naïve Bayes classifiers assume that all features are conditionally independent given the class label. In other words, the presence of one feature is independent of the presence of any other feature, given the class label. Despite this simplifying assumption, Naïve Bayes classifiers often perform well in practice, especially when the assumption is approximately valid or when there's a large amount of data.

3. Model Training: During training, Naïve Bayes classifiers estimate the prior probabilities $P(Y)$ and the class-conditional probabilities $P(X|Y)$ from the training data.

4. Prediction: Given a new instance with features X , Naïve Bayes classifiers use Bayes' theorem to calculate the posterior probabilities $P(Y|X)$ for each class label Y . The class label with the highest posterior probability is then assigned to the instance.

5. Types of Naïve Bayes Classifiers:

Gaussian Naïve Bayes: Assumes that continuous features follow a Gaussian (normal) distribution.

Multinomial Naïve Bayes: Suitable for features that represent counts or frequencies.

Bernoulli Naïve Bayes: Appropriate for binary feature vectors.

Naïve Bayes classifiers are widely used in various applications, including text classification (e.g., spam filtering, sentiment analysis), medical diagnosis, recommendation systems, and more. They are computationally efficient, easy to implement, and can handle large datasets with many features. However, the naïve assumption of feature independence may not hold true in all cases,

which can affect the classifier's performance, especially when dealing with highly correlated features.

6.3.4 Decision Tree

A Decision Tree classifier is a popular machine learning algorithm used for both classification and regression tasks. It's a non-parametric supervised learning method that learns a series of hierarchical decision rules from the feature variables to predict the target variable's class label. Here's how a Decision Tree classifier works:

1. Tree Structure: A Decision Tree is a hierarchical structure consisting of nodes, where each node represents a decision based on a feature variable. The top node is called the root node, and the leaf nodes represent the class labels or regression values.

2. Decision Nodes: Each non-leaf node in the tree represents a decision based on the value of a particular feature variable. The tree splits the dataset into subsets based on these decisions, with each subset becoming increasingly pure with respect to the target variable.

3. Splitting Criteria: To decide which feature to split on and where to split, Decision Trees typically use a measure of impurity or information gain. Common impurity measures include Gini impurity and entropy. The goal is to choose the feature and split point that maximally reduces impurity in the resulting subsets.

4. Recursive Partitioning: The process of splitting the dataset based on feature values continues recursively until one of the stopping criteria is met, such as reaching a maximum tree depth, having a minimum number of samples in a node, or reaching a node where all samples belong to the same class.

5. Prediction: To classify a new instance, it traverses the Decision Tree from the root node down to a leaf node based on the instance's feature values. The class label associated with the leaf node reached is then assigned to the instance.

Decision Trees have several advantages:

Interpretability: Decision Trees are easy to interpret and visualize. The decision rules learned by the tree can be easily understood by humans.

No Assumptions about Data: Decision Trees make no assumptions about the distribution of the data and can handle both numerical and categorical data.

Nonlinear Relationships: Decision Trees can capture nonlinear relationships between features and

the target variable.

However, Decision Trees also have some limitations:

Overfitting: Decision Trees are prone to overfitting, especially when the tree is deep and not pruned properly. Overfitting occurs when the tree captures noise in the training data rather than the underlying patterns.

Instability: Small variations in the data can lead to different tree structures, making Decision Trees unstable.

Bias towards Dominant Classes: Decision Trees tend to favor classes that are dominant in the dataset, leading to biased predictions.

To mitigate these issues, ensemble methods like Random Forests and Gradient Boosted Trees are often used, which combine multiple Decision Trees to improve performance and robustness.

6.3.5 Random Forest

Random Forest is an ensemble learning method that is widely used for classification and regression tasks. It's a powerful and versatile algorithm that builds multiple decision trees during training and combines their predictions to produce more accurate and stable results compared to individual trees. Here's how a Random Forest classifier works:

1. Bootstrap Sampling (Bagging): Random Forest starts by creating multiple bootstrap samples (random samples with replacement) from the original training dataset. Each bootstrap sample is used to train a separate decision tree.

2. Random Feature Selection: At each node of the decision tree, instead of considering all features to determine the best split, Random Forest selects a random subset of features. This introduces randomness and diversity among the trees, making them less correlated.

3. Decision Tree Training: Using the bootstrap samples and the randomly selected features, multiple decision trees are trained independently. Each decision tree is trained recursively by selecting the best split at each node based on a criterion like Gini impurity or entropy.

4. Voting (Classification) or Averaging (Regression): During prediction, each tree in the Random Forest independently classifies the input data point (in case of classification) or predicts its value (in case of regression). For classification tasks, the class with the most votes among the trees is assigned as the final prediction. For regression tasks, the average of the predicted values from all trees is taken as the final prediction.

Random Forest offers several advantages:

High Accuracy: Random Forest generally yields high accuracy in classification and regression tasks, often outperforming individual decision trees.

Robustness to Overfitting: By aggregating the predictions of multiple trees and introducing randomness, Random Forest is less prone to overfitting compared to individual decision trees.

Feature Importance: Random Forest can provide insights into feature importance, allowing users to identify which features are most influential in making predictions.

Scalability: Random Forest can handle large datasets with high dimensionality efficiently.

However, Random Forest also has some limitations:

Model Interpretability: While Random Forest can provide feature importance measures, interpreting the overall decision-making process can be challenging due to the ensemble nature of the algorithm.

Computational Complexity: Training and predicting with Random Forest can be computationally expensive, especially with a large number of trees and features.

Overall, Random Forest is a powerful and versatile algorithm that is widely used in various applications, including classification, regression, and feature selection. It's particularly suitable for tasks where high accuracy and robustness are required.

6.4 Understanding dataset

Understanding a dataset is a crucial step in any data analysis or machine learning task. Here's a systematic approach to understanding a dataset:

1. Obtain the Data: Obtain the dataset you're interested in analyzing. This could be from a variety of sources, such as online repositories, databases, or collected through surveys or experiments.

2. Explore the Data's Structure:

Dimensions: Determine the dimensions of the dataset (number of rows and columns).

Data Types: Identify the types of data present in each column (e.g., numerical, categorical, text).

Column Names: Understand the meaning of each column by examining the column names or headers.

3. Descriptive Statistics:

Summary Statistics: Compute descriptive statistics for numerical columns (e.g., mean, median, standard deviation, minimum, maximum).

Unique Values: Identify unique values and their frequencies for categorical columns.

Missing Values: Check for missing values in the dataset and decide on a strategy for handling them (e.g., imputation, removal).

4. Data Visualization:

Histograms: Plot histograms to visualize the distribution of numerical variables.

Box Plots: Create box plots to identify outliers and understand the spread of the data.

Bar Plots: Generate bar plots to visualize the distribution of categorical variables.

Scatter Plots: Explore relationships between pairs of numerical variables using scatter plots.

Heatmaps: Create heatmaps to visualize correlations between numerical variables.

5. Understanding Relationships:

Correlation Analysis: Compute correlation coefficients to quantify the relationship between numerical variables.

Feature Interactions: Explore how different features interact with each other, especially in predictive modeling tasks.

Domain Knowledge: Leverage domain knowledge to understand the significance of different features and their potential impact on the target variable.

6. Data Cleaning and Preprocessing:

Handling Missing Values: Implement strategies to deal with missing values based on the nature of the dataset and the analysis goals.

Encoding Categorical Variables: Convert categorical variables into numerical format using techniques like one-hot encoding or label encoding.

Feature Scaling: Scale numerical features if necessary to ensure that they have similar ranges.

Feature Engineering: Create new features or transform existing ones to extract more meaningful information.

7. Data Quality Assessment:

Data Consistency: Check for inconsistencies or errors in the data, such as typos or incorrect values.

Data Duplicates: Identify and remove duplicate rows if necessary.

Data Integrity: Ensure that the dataset aligns with expectations and requirements for analysis.

By following these steps, you can gain a comprehensive understanding of the dataset, which forms the foundation for further analysis, modeling, and interpretation.

6.5 Data Preprocessing

Data preprocessing is a crucial step in any data analysis or machine learning project. It involves transforming raw data into a clean, organized format that is suitable for analysis or modeling. Here are some common data preprocessing techniques:

1. Handling Missing Values:

Imputation: Replace missing values with a calculated value (e.g., mean, median, mode) based on the column's distribution.

Deletion: Remove rows or columns with missing values if they are not critical for analysis.

Advanced Techniques: Use more sophisticated methods like interpolation or machine learning algorithms to predict missing values.

2. Encoding Categorical Variables:

One-Hot Encoding: Convert categorical variables into binary vectors, with each category represented by a binary attribute.

Label Encoding: Map each category to a unique numerical value.

Target Encoding: Encode categorical variables based on the target variable's mean or other statistics.

3. Feature Scaling:

Standardization: Scale numerical features to have zero mean and unit variance.

Min-Max Scaling: Scale numerical features to a fixed range, typically between 0 and 1.

Robust Scaling: Scale numerical features using statistics robust to outliers, such as the median and interquartile range.

4. Feature Transformation:

Log Transformation: Transform skewed numerical features to make their distribution more symmetric.

Box-Cox Transformation: Another method for transforming skewed numerical features to make their distribution more normal.

Polynomial Features: Generate polynomial features by combining existing features to capture nonlinear relationships.

5. Handling Outliers:

Trimming: Remove extreme values beyond a certain threshold.

Winsorization: Replace extreme values with a specified percentile value.

Transformation: Apply data transformations like log transformation to mitigate the effect of outliers.

6. Data Normalization:

- Normalize numerical features to scale them within a specific range, typically between 0 and 1.
- Useful for algorithms sensitive to feature scales, such as k-nearest neighbors (KNN) and support vector machines (SVMs).

7. Data Discretization:

- Convert continuous variables into categorical variables by dividing them into bins or intervals.
- Useful for certain algorithms or for reducing the effect of outliers.

8. Handling Imbalanced Classes:

Resampling Techniques: Over-sample the minority class or under-sample the majority class to balance class distribution.

Algorithmic Approaches: Use algorithms that are robust to class imbalance, such as ensemble methods like Random Forest or Gradient Boosting.

9. Feature Selection:

Identify and select the most relevant features for modeling, using techniques like correlation analysis, feature importance ranking, or dimensionality reduction methods like principal component analysis (PCA).

By performing these preprocessing steps, you can ensure that your data is clean, standardized, and ready for analysis or modeling, leading to more accurate and robust results.

6.6 Source Code

```
# Step 1: Import relevant libraries-----
```

```
#Standard libraries for data analysis:-----
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.stats import norm, skew
from scipy import stats
import statsmodels.api as sm
```


#Standard libraries for data visualization-----

```
import seaborn as sn
from matplotlib import pyplot
import matplotlib.pyplot as plt
import matplotlib.pylab as pylab
import matplotlib
%matplotlib inline
color = sn.color_palette()
import matplotlib.ticker as mtick
from IPython.display import display
pd.options.display.max_columns = None
from pandas.plotting import scatter_matrix
```

#Miscellaneous Utilitiy Libraries-----

```
import random
import os
import re
import sys
import timeit
import string
import time
from datetime import datetime
from time import time
from dateutil.parser import parse
```

#Step 2: Set up current working directory-----

```
from google.colab import drive
drive.mount('/content/drive')
```

```

# Step 3: Import the dataset-----
dataset =pd.read_csv("drive/My Drive/Colab Notebooks/customer_churn_data.csv")

# Step 4: Evaluate Datastructure -----
dataset.head()

#Step 5: Check Target Variable Distribution -----
dataset["Churn"].value_counts()

#Step 6: Clean the Dataset-----
dataset["TotalCharges"] = pd.to_numeric(dataset["TotalCharges"],errors='coerce')
dataset["TotalCharges"] = dataset["TotalCharges"].astype("float")

#*Find the average and fill missing values of each columns programmatically.
na_cols = dataset.isna().any()
na_cols = na_cols[na_cols == True].reset_index()
na_cols = na_cols["index"].tolist()
for col in dataset.columns[1:]:
    if col in na_cols:
        if dataset[col].dtype != 'object':
            dataset[col] = dataset[col].fillna(dataset[col].mean()).round(0)

#Step 8: label Encode Binary data-----
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
#Create a label encoder object
le = LabelEncoder()
# Label Encoding will be used for columns with 2 or less unique values
le_count = 0

```

```

for col in dataset.columns[1:]:
    if dataset[col].dtype == 'object':
        if len(list(dataset[col].unique())) <= 2:
            le.fit(dataset[col])
            dataset[col] = le.transform(dataset[col])
            le_count += 1
print('{} columns were label encoded.'.format(le_count))

```

#Step 9: Exploratory Data Analysis-----

```

dataset2 = dataset[['gender', 'SeniorCitizen', 'Partner', 'Dependents',
                    'tenure', 'PhoneService', 'PaperlessBilling',
                    'MonthlyCharges', 'TotalCharges']]
fig = plt.figure(figsize=(15, 12))
plt.suptitle('Histograms of Numerical Columns\n',horizontalalignment="center",fontstyle =
"normal", fontsize = 24, fontfamily = "sans-serif")
for i in range(dataset2.shape[1]):
    plt.subplot(6, 3, i + 1)
    f = plt.gca()
    f.set_title(dataset2.columns.values[i])
    vals = np.size(dataset2.iloc[:, i].unique())
    if vals >= 100:
        vals = 100
    plt.hist(dataset2.iloc[:, i], bins=vals, color = '#1b8f90')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])

```

#Step 9.5. Plot Correlation Matrix of all independent variables-----

```

## Set and compute the Correlation Matrix
sn.set(style="white")
corr = dataset2.corr()

```

```

# Set up the matplotlib figure and a diverging colormap
f, ax = plt.subplots(figsize=(12, 10))

# Draw the heatmap with the mask and correct aspect ratio
sn.heatmap(corr, cmap='BuPu', annot=True, square=True, linewidths=2, xticklabels=False)

#Step 12: Generate training and test datasets of dependent and independent variables-----
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(dataset, response, stratify=response, test_size =
0.2, random_state = 0)

print("Number transactions X_train dataset: ", X_train.shape)
print("Number transactions y_train dataset: ", y_train.shape)
print("Number transactions X_test dataset: ", X_test.shape)
print("Number transactions y_test dataset: ", y_test.shape)

# Step 14: Feature Scaling-----
from sklearn.preprocessing import StandardScaler

sc_X = StandardScaler()

X_train2 = pd.DataFrame(sc_X.fit_transform(X_train))

X_train2.columns = X_train.columns.values
X_train2.index = X_train.index.values

X_train = X_train2

X_test2 = pd.DataFrame(sc_X.transform(X_test))

X_test2.columns = X_test.columns.values
X_test2.index = X_test.index.values

X_test = X_test2

#Step 15.1: Compare Baseline Classification Algorithms - First Iteration
#Using Accuracy and ROC AUC Mean Metrics

from sklearn.linear_model import LogisticRegression

```

```

from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

models = []

models.append(('Logistic Regression', LogisticRegression(solver='liblinear', random_state = 0,
                                                         class_weight='balanced')))

models.append(('SVC', SVC(kernel = 'linear', random_state = 0)))

models.append(('Gaussian NB', GaussianNB()))

models.append(('Decision Tree Classifier',
               DecisionTreeClassifier(criterion = 'entropy', random_state = 0)))

models.append(('Random Forest', RandomForestClassifier(
    n_estimators=100, criterion = 'entropy', random_state = 0)))

#Evaluating Model Results:

acc_results = []
auc_results = []
names = []

# set table to table to populate with performance results
col = ['Algorithm', 'ROC AUC Mean', 'ROC AUC STD', 'Accuracy Mean', 'Accuracy STD']
model_results = pd.DataFrame({"Algorithm":pd.Series(dtype='str'),
                              "ROC AUC Mean" : pd.Series (dtype='float'),
                              "ROC AUC STD" : pd.Series (dtype='float'),
                              "Accuracy Mean" : pd.Series (dtype='float'),
                              "Accuracy STD" : pd.Series (dtype='float')})

from sklearn import model_selection

i = 0

# evaluate each model using k-fold cross-validation

```

```

for name, model in models:

    kfold = model_selection.KFold(
        n_splits=10) # 10-fold cross-validation
    cv_acc_results = model_selection.cross_val_score( # accuracy scoring
        model, X_train, y_train, cv=kfold, scoring='accuracy')
    cv_auc_results = model_selection.cross_val_score( # roc_auc scoring
        model, X_train, y_train, cv=kfold, scoring='roc_auc')
    acc_results.append(cv_acc_results)
    auc_results.append(cv_auc_results)
    names.append(name)
    model_results.loc[i] = [name,
        round(cv_auc_results.mean()*100, 2),
        round(cv_auc_results.std()*100, 2),
        round(cv_acc_results.mean()*100, 2),
        round(cv_acc_results.std()*100, 2)
    ]

    i += 1

model_results.sort_values(by=['ROC AUC Mean'], ascending=False)

```

CHAPTER-7

SYSTEM TESTING

7. SYSTEM TESTING

7.1 Introduction

A test case is a specification of the inputs, execution conditions, testing procedure, and expected results that define a single test to be executed to achieve a particular software testing objective. The mechanism for determining whether a software program or system has passed or failed such a test is known as a test oracle. In some setting, an oracle could be a requirement or use case. It may take a test case to determine that a software program or system is functioning correctly. Test cases are often referred to as test scripts, particularly when written. Written test cases are usually collected into test suites.

7.1.1 Levels of Testing

In order to uncover the errors, present in different phases, we have the concept of levels of testing. The basic levels of testing are

- Client needs Acceptance Testing
- System Testing
- Design Integration Testing
- Code Unit Testing

7.1.2 Software Testing Strategies

- Unit Testing
- Integration Testing
- Validation Testing
- System Testing

Unit Testing

Unit Testing is a level of software testing where individual units/components of the software are tested. The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software. It usually has one or a few inputs and usually a single output. In procedural programming, a unit may be an individual program, function, procedure, etc. In object-oriented programming, the smallest unit is a method, which may belong

to a base/ super class, abstract class or derived/ child class. (Some treat a module of an application as a unit. This is to be discouraged as there will probably be many individual units within that module.) Unit testing frameworks, drivers, stubs, and mock/ fake objects are used to assist in unit testing.

Integration Testing

Integration Testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. Test drivers and test stubs are used to assist in integration Testing. Integration Testing is the second level of testing performed after Unit Testing and before System Testing.

Validation Testing

The process of evaluating software during the development process or at the end of the development process to determine whether it satisfies specified business requirements. Validation Testing ensures that the product actually meets the client's needs. It can also be defined as to demonstrate that the product fulfills its intended use when deployed in an appropriate environment.

System Testing

System testing is a level of software testing where a complete and integrated software is tested. The purpose of this test is to evaluate the system's compliance with the specified requirements. System testing is performed on the entire system in the context of either functional requirement specifications (FRS) or system requirement specification (SRS), or both. System testing tests not only the design but also the behavior and even the believed expectations of the customer. It is also intended to test up to and beyond the bounds defined in the software or hardware requirements specification.

7.2 Test Cases

Test Case Id	Input Validation	Non-numeric values	Error message	Error message	Pass
TC001	Input Validation	Non-numeric values	Error message	Error message	Pass
TC002	Training	Sample training dataset	Trained model	Trained model	Pass
TC003	Prediction Accuracy	Test dataset with known outcomes	Predicted outcomes	Predicted outcomes	Pass
TC004	Performance	Precision, recall, F1 score calculation	Precision, recall, F1 score values	Calculated values	Pass
TC005	Generalization	Validation dataset	Predicted outcomes	Predicted outcomes	Pass
TC006	Robustness	Noisy data	Predicted outcomes	Predicted outcomes	Pass
TC007	Scalability	Large dataset	Predicted outcomes	Predicted outcomes	Pass
TC008	Bias and Fairness	Dataset with bias	Fair predictions	Fair predictions	Pass
TC009	Interpretability	Explain predictions	Explanation	Explanation	Pass
TC010	Integration	Integration with external system	Successful integration	Successful integration	Pass

CHAPTER-8

SCREENSHOTS

8. SCREENSHOTS

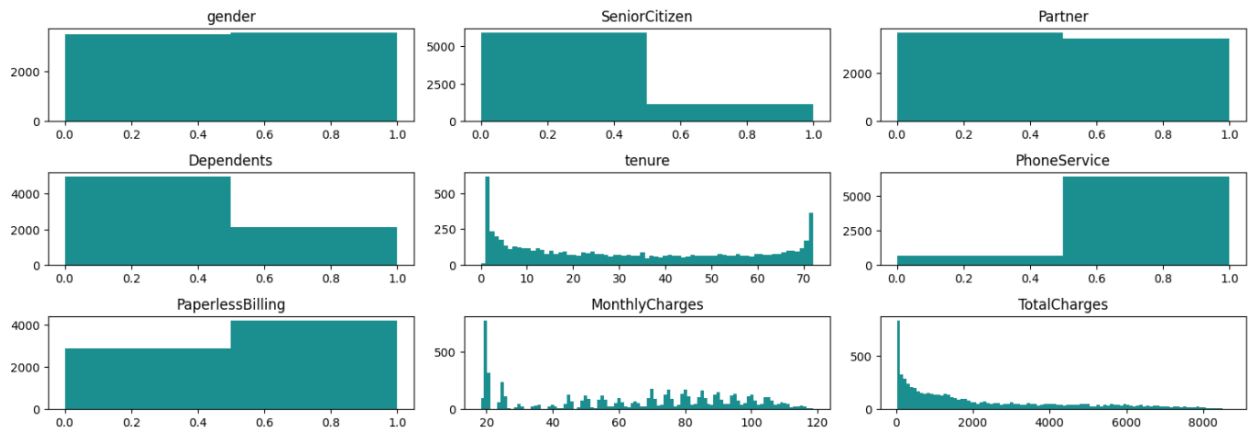
	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

8.1. Description of the dataset

```
customerID      False
gender          False
SeniorCitizen   False
Partner         False
Dependents      False
tenure          False
PhoneService    False
MultipleLines   False
InternetService False
OnlineSecurity  False
OnlineBackup    False
DeviceProtection False
TechSupport     False
StreamingTV     False
StreamingMovies False
Contract        False
PaperlessBilling False
PaymentMethod   False
MonthlyCharges  False
TotalCharges    False
Churn           False
dtype: bool
```

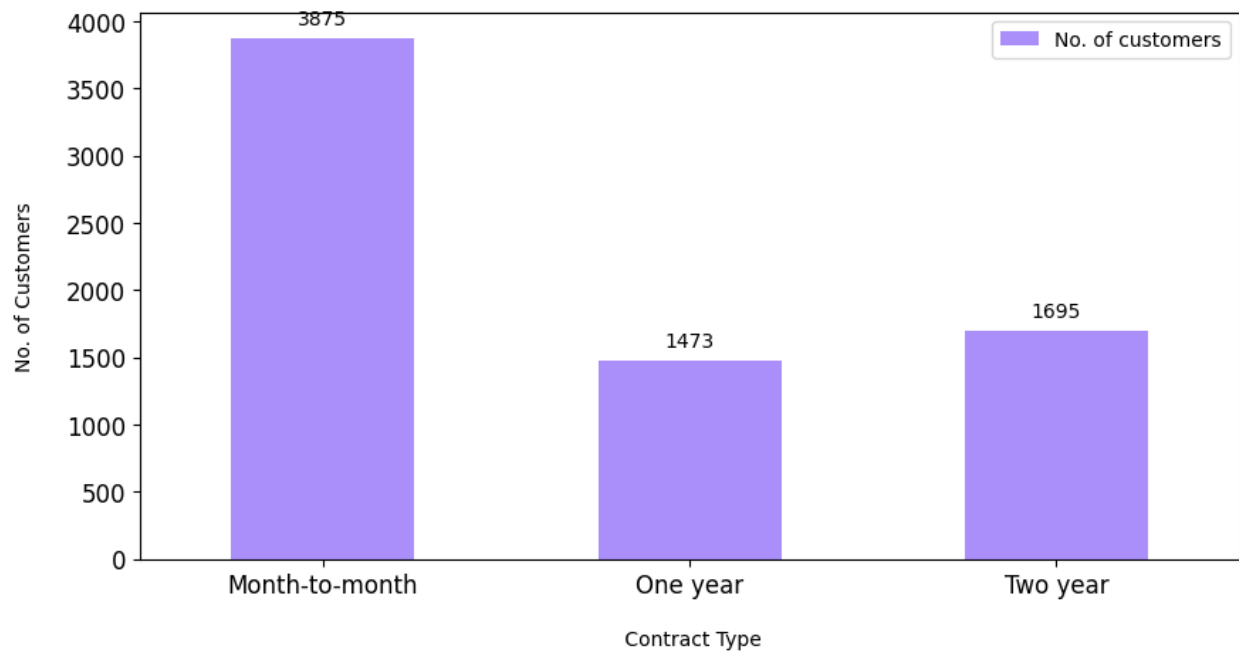
8.2 NULL features details

Histograms of Numerical Columns

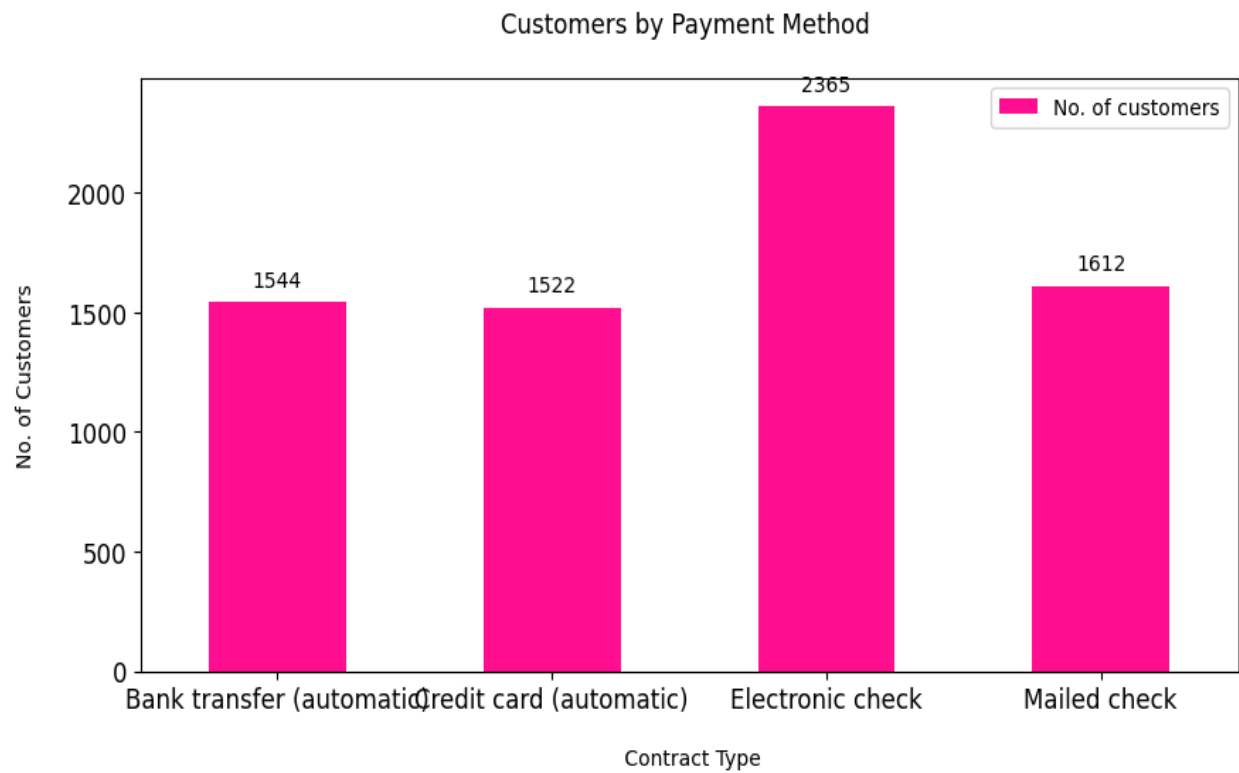


8.3 Histograms of Numerical features

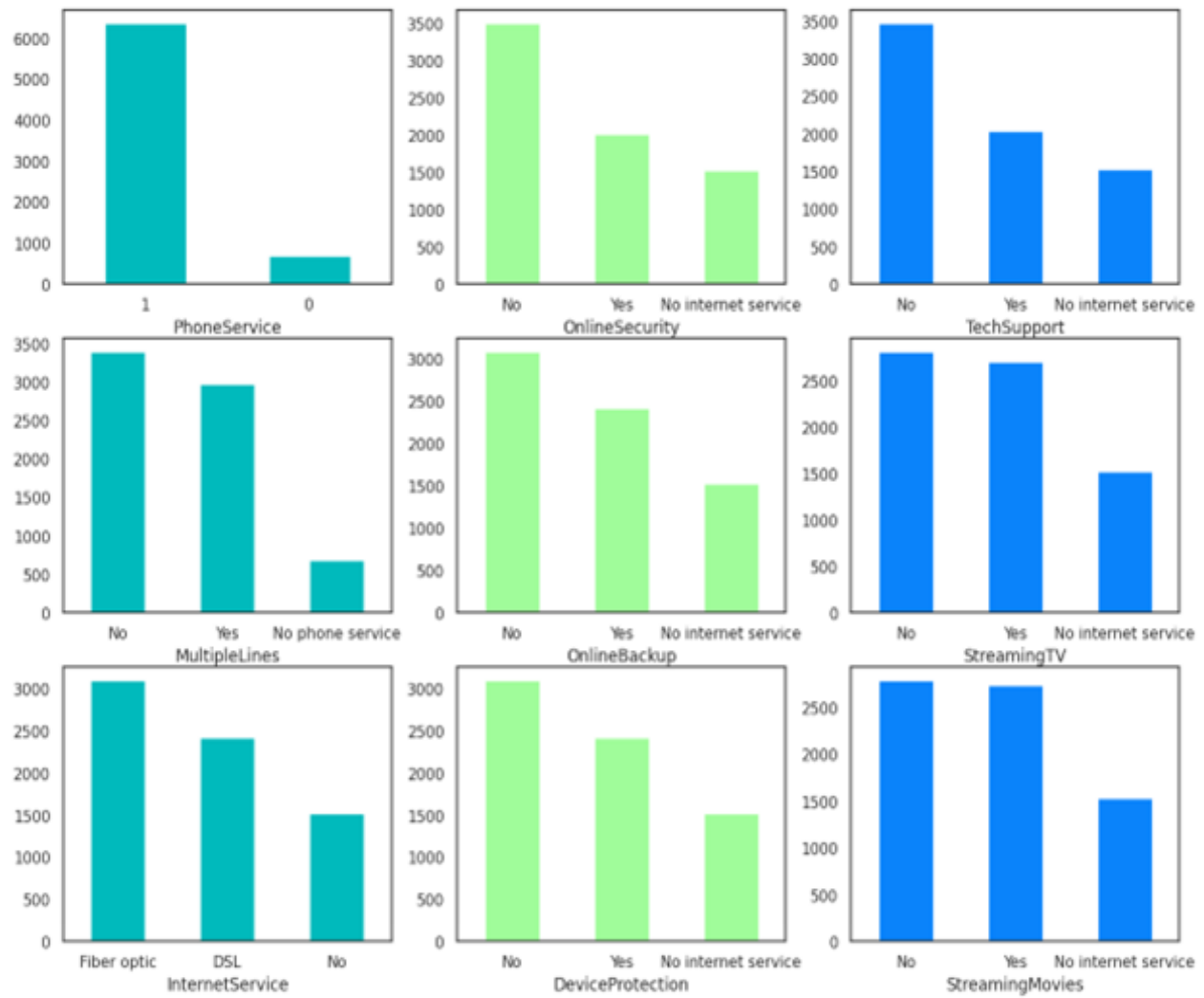
Customers by Contract Type



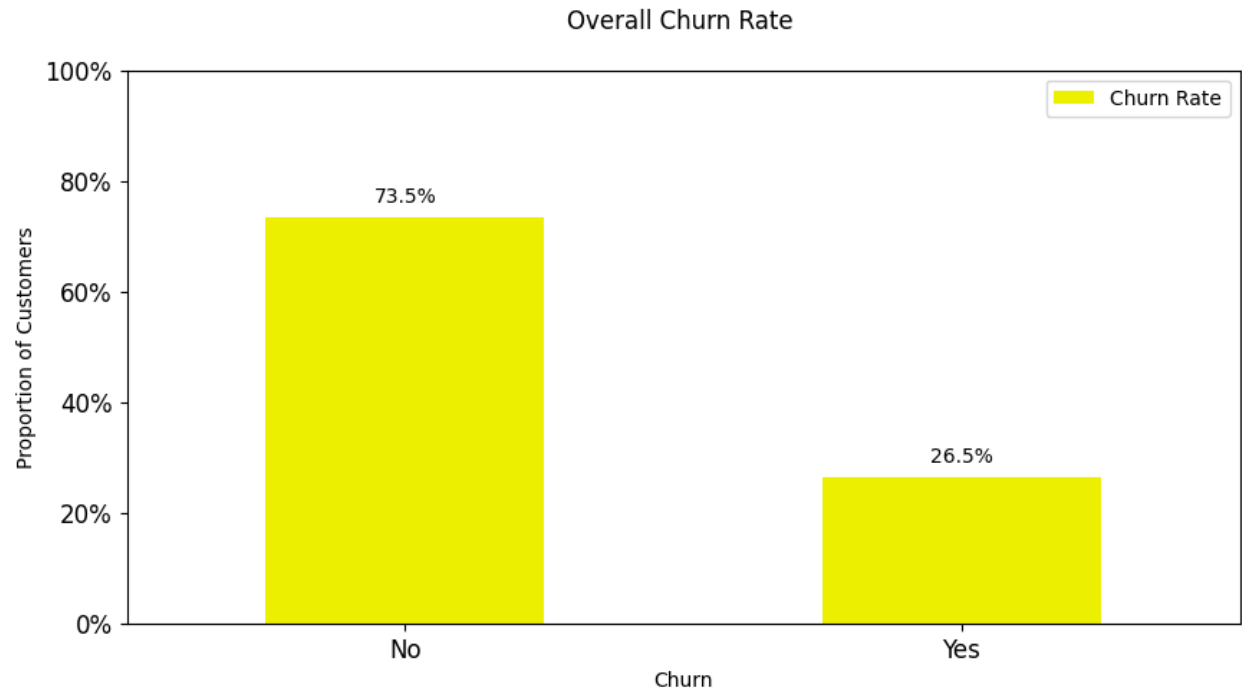
8.4 Customers by Contract type



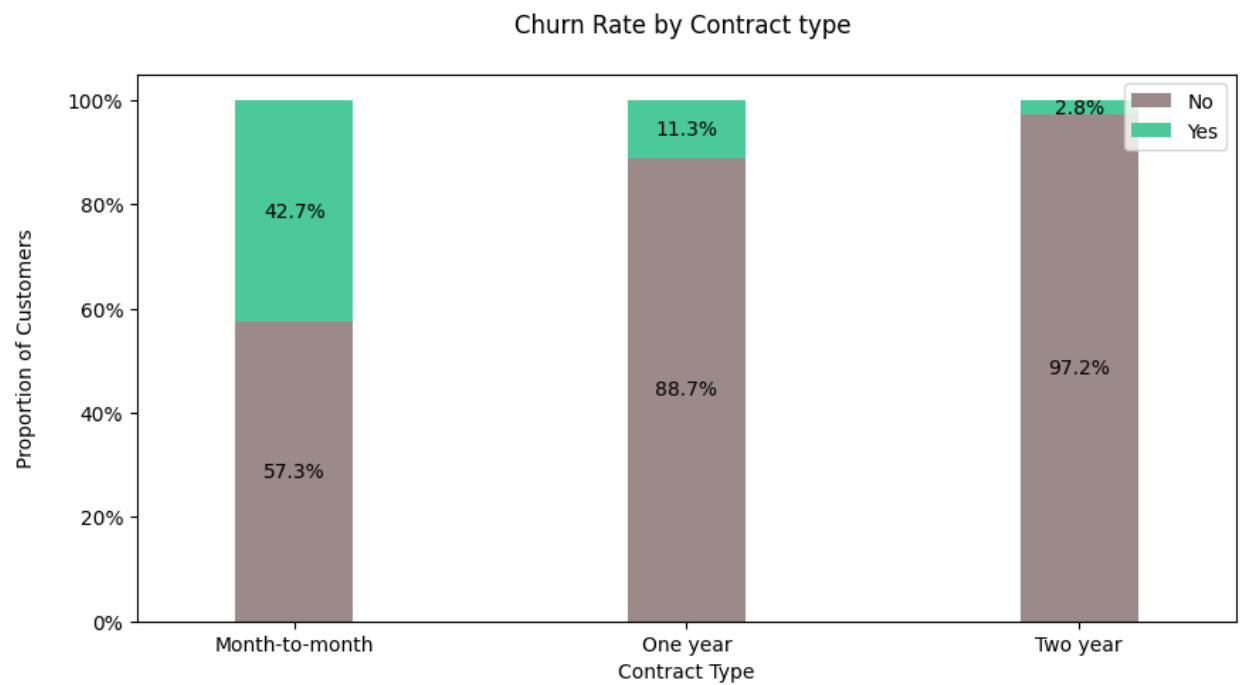
8.5 Customers by Payment method feature



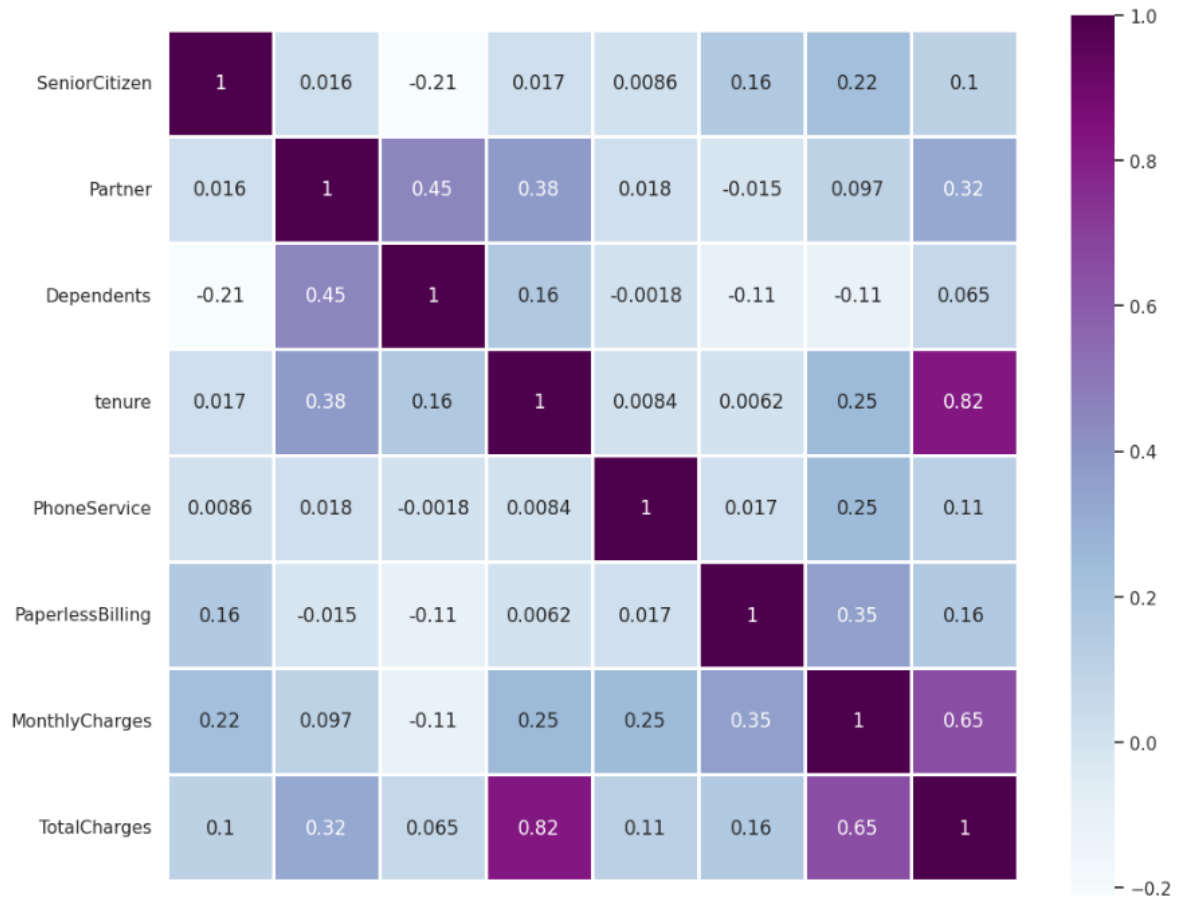
8.6 Distribution of various Label Encoded Categorical Variables



8.7 Overall Churn rate



8.8 Churn Rate by Contract Type



8.9 Correlation heatmap

variables			VIF	variables			VIF
0	gender		1.921286	0	gender		1.879536
1	SeniorCitizen		1.327766	1	SeniorCitizen		1.323089
2	Partner		2.815272	2	Partner		2.814574
3	Dependents		1.921208	3	Dependents		1.908533
4	tenure		10.549667	4	tenure		3.287603
5	PhoneService		7.976386	5	PhoneService		5.963240
6	PaperlessBilling		2.814160	6	PaperlessBilling		2.745897
7	MonthlyCharges		13.988649	7	MonthlyCharges		7.453993
8	TotalCharges		12.570269				

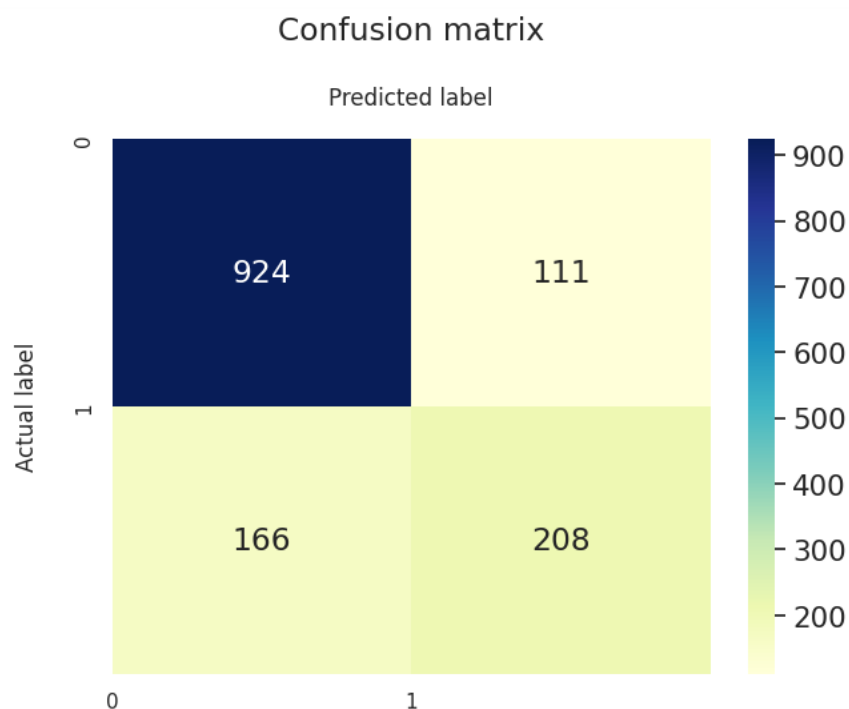
8.10 Multicollinearity using Variance Inflation Factor (VIF) before and after removing TotalCharges

	Algorithm	ROC AUC Mean	ROC AUC STD	Accuracy Mean	Accuracy STD
0	Logistic Regression	84.12	1.65	74.60	1.26
1	SVC	83.64	1.68	79.98	1.08
2	Gaussian NB	81.82	1.79	68.99	1.46
4	Random Forest	81.72	2.02	78.47	1.57
3	Decision Tree Classifier	65.44	1.67	72.75	1.45

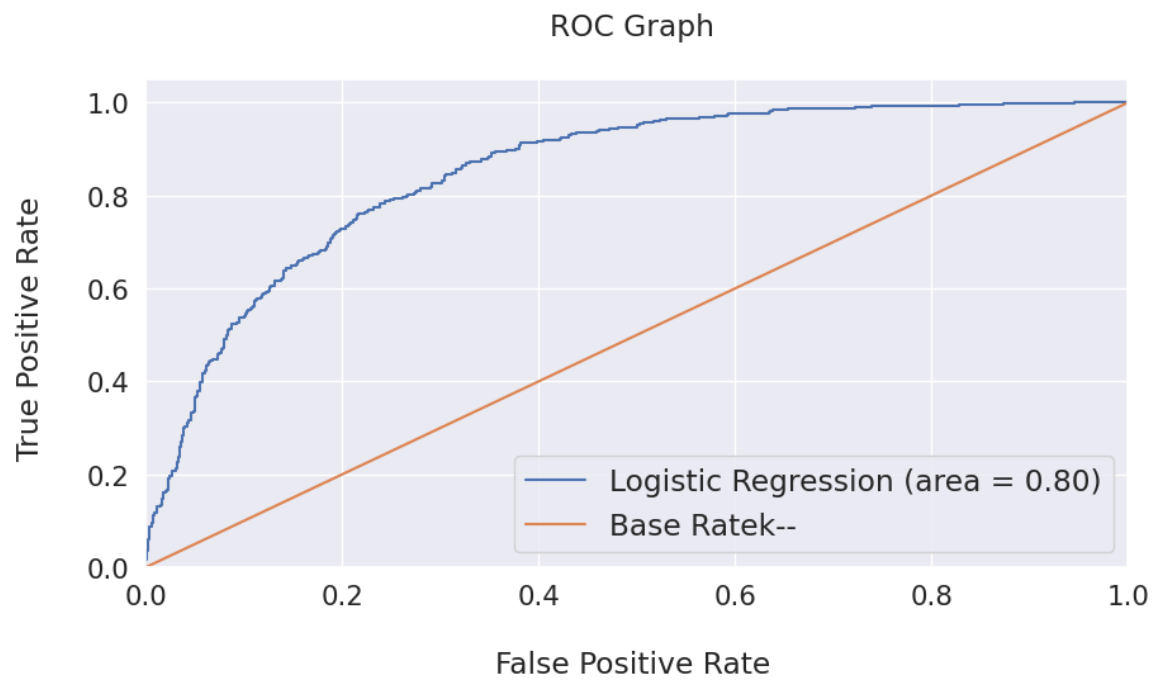
8.11 Baseline Classification Algorithms - Using Accuracy and ROC AUC Mean Metrics

	Model	Accuracy	Precision	Recall	F1 Score	F2 Score
0	Logistic Regression	0.803407	0.652038	0.556150	0.600289	0.573003
1	SVM (Linear)	0.803407	0.650155	0.561497	0.602582	0.577240
4	Random Forest	0.777857	0.611722	0.446524	0.516229	0.472018
3	Decision Tree	0.739532	0.508997	0.529412	0.519004	0.525199
2	Naive Byes	0.703336	0.467359	0.842246	0.601145	0.725806

8.12 Visualize the results and compare the baseline algorithms on Test data



8.13 Confusion Matrix for Logistic Regression



8.14 ROC Graph for Logistic Regression

CHAPTER-9

SMOTE ANALYSIS

9. SMOTE ANALYSIS

9.1 Introduction

SMOTE (synthetic minority oversampling technique) is one of the most commonly used oversampling methods to solve the imbalance problem.

It aims to balance class distribution by randomly increasing minority class examples by replicating them.

SMOTE synthesizes new minority instances between existing minority instances.

It generates the virtual training records by linear interpolation for the minority class.

These synthetic training records are generated by randomly selecting one or more of the k-nearest neighbors for each example in the minority class.

After the oversampling process, the data is reconstructed and several classification models can be applied for the processed data.

9.2 Working of SMOTE analysis

Step 1: Setting the minority class set A, for each $x \in A$, the k-nearest neighbors of x are obtained by calculating the Euclidean distance between x and every other sample in set A.

Step 2: The sampling rate N is set according to the imbalanced proportion. For each $x \in A$, N examples (i.e $x_1, x_2, \dots, x_n$) are randomly selected from its k-nearest neighbors, and they construct the set A_1

Step 3: For each example $x_k \in A_1$, ($k=1, 2, 3 \dots N$), the following formula is used to generate a new example:

$$x' = x + \text{rand}(0, 1) * |x - x_k|$$

in which $\text{rand}(0, 1)$ represents the random number between 0 and 1.

```
# Count no. of 0, 1 in y_train to check the problem is balanced or not

a = np.array(y_train.tolist())
a= a.tolist()
print (a.count(0))
print (a.count(1))
```

4139

4139

9.1 Count of 0, 1 in y_train to check the problem is balanced or not

	Model	Accuracy	Precision	Recall	F1 Score	F2 Score
3	Decision Tree	0.997946	0.999758	0.996134	0.997943	0.996857
4	Random Forest	0.997946	0.998067	0.997826	0.997946	0.997874
0	Logistic Regression	0.776033	0.761024	0.804784	0.782292	0.795634
1	SVM (Linear)	0.746920	0.707302	0.842474	0.768993	0.811459
2	Naive Byes	0.748128	0.704337	0.855279	0.772504	0.820128

9.2 Baseline Classification Algorithms - Using Accuracy and ROC AUC Mean Metrics after performing SMOTE oversampling technique

	Model	Accuracy	Precision	Recall	F1 Score	F2 Score
4	Random Forest	0.803865	0.837809	0.753623	0.793489	0.769079
3	Decision Tree	0.747343	0.778261	0.691787	0.732481	0.707510
0	Logistic Regression	0.787923	0.769928	0.821256	0.794764	0.810450
2	Naive Byes	0.761836	0.718196	0.861836	0.783487	0.828688
1	SVM (Linear)	0.745894	0.709810	0.831884	0.766014	0.804222

9.3 Visualize the results and compare the baseline algorithms on Test data

CHAPTER-10

CONCLUSION AND

FUTURE ENHANCEMENTS

10. CONCLUSION AND FUTURE ENHANCEMENTS

For contemporary businesses operating under a subscription-based model, effectively managing client churn, especially through predictive analysis, has become imperative. Thus, this study aimed to devise a system capable of accurately forecasting customer churn within businesses. Our research reveals that machine learning offers a promising avenue for predicting customer attrition with a notable level of precision. By delving into various aspects of customer churn prediction using machine learning, we strive to enhance the comprehension of how machine learning techniques can be harnessed for this purpose.

In this study, we employed five classification algorithms on a customer dataset to forecast customer churn. Our model pipeline entails several data preprocessing stages, including data visualization for insight generation, feature selection based on correlation and variance inflation factor (VIF), and addressing class imbalance. The dataset is partitioned into training and testing subsets with an 80:20 ratio, followed by data normalization prior to applying machine learning algorithms. The classification algorithms utilized in our analysis encompass *Logistic Regression*, *Support Vector Classifier*, *Gaussian Naïve Bayes*, *Decision Tree Classifier*, and *Random Forest*. We evaluated the performance of these algorithms using various metrics such as ROC AUC Mean, ROC AUC STD, Accuracy Mean, Accuracy STD, Accuracy, Precision, Recall, F1 Score, and F2 Score.

Boosting algorithms hold significant importance in the realm of machine learning, particularly for classification problems, due to several key advantages. Boosting algorithms play a crucial role in classification problems in machine learning by offering enhanced accuracy, robustness, versatility, interpretability, and adaptability, making them indispensable tools for data scientists and practitioners alike. So, this problem can be further studied with boosting algorithms to handle the overfitting of model towards the train data comparing with test data.

CHAPTER-11

REFERENCES

11. REFERENCES

- [1] N. V. Chimankpam and D. Kamil, “Customer churn prediction for business intelligence using machine learning,” in *2021 3rd International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA)*, June 2021, pp. 1–7.
- [2] D. Soumi, P. Prabu, and P. Joy, “Effective ml techniques to predict customer churn,” in *2021 Third International Conference on Inventive Research in Computing Applications (ICIRCA)*, 2021, pp. 895–902.
- [3] S.-Y. Hung, D. C. Yen, and H.-Y. Wang, “Applying data mining to telecom churn management,” *Expert Systems with Applications*, vol. 31, no. 3, pp. 515–524, 2006.
- [4] E. Ngai, L. Xiu, and D. Chau, “Application of data mining techniques in customer relationship management: A literature review and classification,” *Expert Systems with Applications*, vol. 36, no. 2, Part 2, pp. 2592–2602, 2009.
- [5] M. Debajyoti, M. Aarati, N. Anagha, and S. Shriya, “An approach to mitigate the risk of customer churn using machine learning algorithms,” in *Machine Learning for Predictive Analysis*, J. Amit, K. Mahdi, and G. Neeraj, Eds. Singapore: Springer Singapore, 2021, pp. 133–142.
- [6] N.V.Chawla, K.W.Bowyer, L.O.Hall, and W.P.Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [7] R. M. Pereira, Y. M. Costa, and C. N. Silla Jr., “Mtl: A multi-label approach for the tomek link undersampling algorithm,” *Neurocomputing*, vol. 383, pp. 95–105, 2020.
- [8] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, “A study of the behavior of several methods for balancing machine learning training data,” *SIGKDD Explor. Newsl.*, vol. 6, no. 1, p. 20–29, jun 2004.
- [9] W. Verbeke, D. Martens, C. Mues, and B. Baesens, “Building comprehensible customer churn prediction models with advanced rule induction techniques,” *Expert Systems with Applications*, vol. 38, no. 3, pp. 2354–2364, 2011.
- [10] K. Coussement, D. F. Benoit, and D. Van den Poel, “Improved marketing decision making in a customer churn prediction context using generalized additive models,” *Expert Systems with Applications*, vol. 37, no. 3, pp. 2132–2143, 2010.

- [11] N. N. A. Sjarif, M. R. M. Yusof, D. H. Wong¹, S. Ya'akob¹, R. Ibrahim¹, and M. Z. Osman, "A customer churn prediction using pearson correlation function and k nearest neighbor algorithm for telecommunication industry," *Int. J. Advance Soft Compu. Appl*, vol. 11, no. 2, 2019.
- [12] J. Burez and D. Van den Poel, "Crm at a pay-tv company: Using analytical models to reduce customer attrition by targeted marketing for subscription services," *Expert Systems with Applications*, vol. 32, no. 2, pp. 277–288, 2007.
- [13] J. Hadden, A. Tiwari, R. Roy, and D. Ruta, "Computer assisted customer churn management: State-of-the-art and future trends," *Computers & Operations Research*, vol. 34, no. 10, pp. 2902–2917, 2007.
- [14] T. C. S. Renato Alexandre de Lima Lemos and B. M. Tabak, "Propension to customer churn in a financial institution: a machine learning approach," *Neural Computing and Applications*, 2022.
- [15] N. N. Vo, S. Liu, X. Li, and G. Xu, "Leveraging unstructured call log data for customer churn prediction," *Knowledge-Based Systems*, vol. 212, p. 106586, 2021.
- [16] S. Animesh, "Application of machine learning and statistics in banking customer churn prediction," in *2021 8th International Conference on Smart Computing and Communications (ICSCC)*, July 2021, pp. 37–41.